MIT School of Computing
Department of Computer Science & Engineering

MIT-ADT UNIVERSITY
PUNE, INDIA

23CSE2018 – Foundations of Artificial Intelligence

Class -S.Y. (SEM-II), AIA

Unit - II PROBLEM SOLVING AND SEARCH STRATEGIES

Coordinator, Co-Coordinator

Prof. Dr. Sumit Hirve

Prof. Aarti Pimpalkar

Team Members

Prof. Jyoti Gavhane
Prof. Amol Chincholkar
Prof. Nitin Alzende
Prof. Komal Munde
Prof. Dr. Rajendra Pawar
Prof. Sushma Mehetre
Prof. Dr. Mayura Shelke
Prof. Sneha Singha
Prof. Swati Powar
Prof. Dr. Tejaswini Bhosale
Prof. Sneha Deshmukh
Prof. Rasmhi Tuptewar
Prof. Swati Kadam

AY 2025-26 SEM-II

Unit II - Syllabus

Unit II: PROBLEM SOLVING AND SEARCH STRATEGIES : (09 hours)

- Problem-Solving Agents and Formulating Problems, Toy Problems,
- Uninformed Search Strategies: Breadth First Search, Depth First Search, Depth Limited Search, Uniform Cost Search, Iterative Deepening Search, Bi-directional Search,
- Informed Search Strategies: Best first Search, A* Search, AO* Search, Greedy Search, Hill Climbing,
- Adversarial search: Minimax, Alpha-Beta Pruning

Problem space and search

PROBLEM SOLVING AGENT (GOAL BASED)

- What is the Problem?
- Decide what to do by finding sequences of actions that lead to desirable states.
- What can be Solution or Solutions ?
- General Purpose Search Algorithm to solve problem- analysis of algorithms
- Goal formulation, based on the current situation and the agent's performance measure, is the **first step in problem solving**

Problem space and search

- An agent with several immediate options of **unknown value** can decide
- what to do by just examining different possible sequences of actions that **lead to states of known value**, and **then choosing the best sequence**.

This process of looking for such a sequence is called **Search**

Simple steps for achieving goal

"Formulate, Search, Execute"

Problem space and search

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  inputs: percept, a percept
  static: seq, an action sequence, initially empty
          state, some description of the current world state
          goal, a goal, initially null
          problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then do
      goal ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq ← SEARCH(problem)
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Figure 3.1 A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over. Note that when it is executing the sequence it ignores its percepts: it assumes that the solution it has found will always work.

Environment is Static, observable, deterministic. Initial state known

Problem space and search

- The problem can then be **solved** by using the rules, in combination with an appropriate control strategy, to move through the **problem space** until a path from an initial state to a goal state is found.
- This process is known as **Search**.

PROBLEMS SOLVING STEPS

To solve the problem of building a system you **should take the following steps**:

- 1. Define the problem accurately including detailed specifications and what constitutes a suitable solution.
- 2. Scrutinize the problem carefully, for **some features may have a central affect** on the chosen method of solution.
- 3. Segregate and represent the **background knowledge needed** in the solution of the problem.
- 4. **Choose** the best solving techniques for the problem to solve a solution..

PROBLEMS SOLVING STEPS

- Problem solving is a process of generating solutions from observed data.
- characterized by a set of goals, objects, and operations.
- **Problem space** is an abstract space.
- A problem space encompasses all valid states that can be generated by the application of any combination of operators on any combination of objects.
- The problem space may contain one or more **solutions**.
- A **solution** is a combination of operations and objects that achieve the goals.

State Space Search

A state space represents a problem in terms of **states and operators** that change states.

A state space consists of:

- A representation of the states the system can be in.

  ex.in a board game, the board represents the current state of the game.

- A set of **operators** that can change one state into another state.

  In a board game, the operators are the legal moves from any given state,

  represented as programs that change a state representation to represent the new state.

- An initial state.
- A set of final states;

  some of these may be desirable, others undesirable.

The Water Jug Problem

In this problem, we use two jugs called four and three; four holds a maximum of four gallons of water and three a maximum of three gallons of water.

How can we get two gallons of water in the four jug?

The Water Jug Problem

- The state space is a set of **prearranged pairs** giving the number of gallons of water in the pair of jugs at any time, i.e., (four, three) where four = 0, 1, 2, 3 or 4 and three = 0, 1, 2 or 3.
- The start state is (0, 0) and the goal state is (2, n) where n may be any but it is limited to three holding from 0 to 3 gallons of water or empty.
- Three and four shows the name and numerical number shows the amount of water in jugs for solving the water jug problem.
- The major production rules for solving this problem are .

The Water Jug Problem

Initial condition	Goal comment
1. (four, three) if four < 4	(4, three) fill four from tap
2. (four, three) if three< 3	(four, 3) fill three from tap
3. (four, three) If four > 0	(0, three) empty four into drain
4. (four, three) if three > 0	(four, 0) empty three into drain
5. (four, three) if four + three<4	(four + three, 0) empty three into four
6. (four, three) if four + three<3	(0, four + three) empty four into three
7. (0, three) If three > 0	(three, 0) empty three into four
8. (four, 0) if four > 0	(0, four) empty four into three
9. (0, 2)	(2, 0) empty three into four
10. (2, 0)	(0, 2) empty four into three
11. (four, three) if four < 4	(4, three-diff) pour diff, 4-four, into four from three
12. (three, four) if three < 3	(four-diff, 3) pour diff, 3-three, into three from four and a solution is given below four three rule

*(**Fig. 2.2** Production Rules for the Water Jug Problem)*

The Water Jug Problem

Gallons in Four Jug	Gallons in Three Jug	Rules Applied
0	0	-
0	3	2
3	0	7
3	3	2
4	2	11
0	2	3
2	0	10

- The problem solved by using the production rules in combination with an appropriate control strategy, **moving through the problem space** until a path from an initial state to a goal state is found.
- In this problem solving process, search is the fundamental concept.
- For simple problems it is easier to achieve this goal by hand but there will be cases where this is far too difficult.

TOY PROBLEMS

The vacuum world

- **States:** The agent is in one of two locations, each of which might or might not contain dirt. Thus there are 2 x 2^2 = 8 possible world states.
- **Initial state:** Any state can be designated as the initial state.
- **Successor function:** This generates the legal states that result from trying the three actions (Left, Right, and Suck). The complete state space is shown in
- **Goal test:** This checks whether all the squares are clean.
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

TOY PROBLEMS

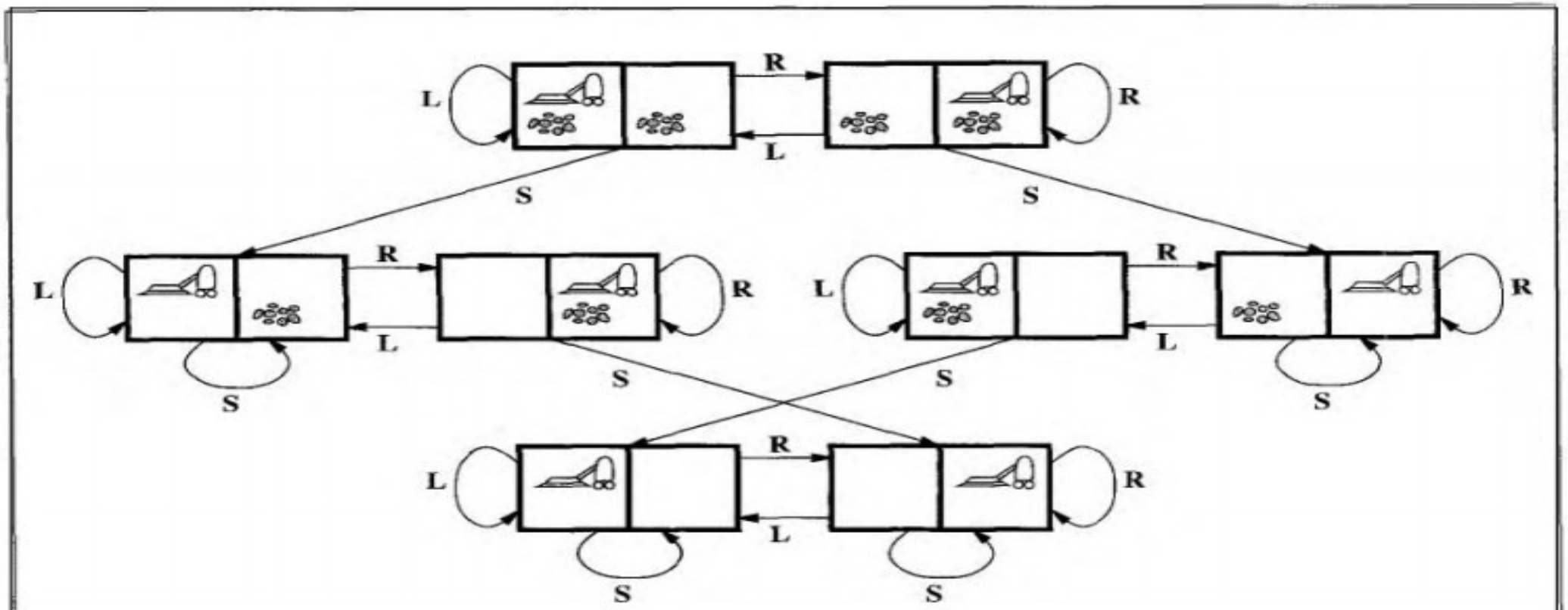


Figure 3.3 The state space for the vacuum world. Arcs denote actions: L = *Left*, R = *Right*, S = Suck.

THE 8-PUZZLE

- Consists of a 3 x 3 board with eight numbered tiles and a blank space
- A tile adjacent to the blank space can slide into the space
- Object is to reach a specified goal state, such as the one shown on the right of the figure
- **States:** A state description specifies the location of each of the eight tiles and the blank in one of the nine squares
- **Initial state:** Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states
- **Successor function:** This generates the legal states that result from trying the four actions (blank moves Left, Right, Up, or Down).
- **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.
- **Goal test:** This checks whether the state matches the goal configuration shown in

THE 8-PUZZLE

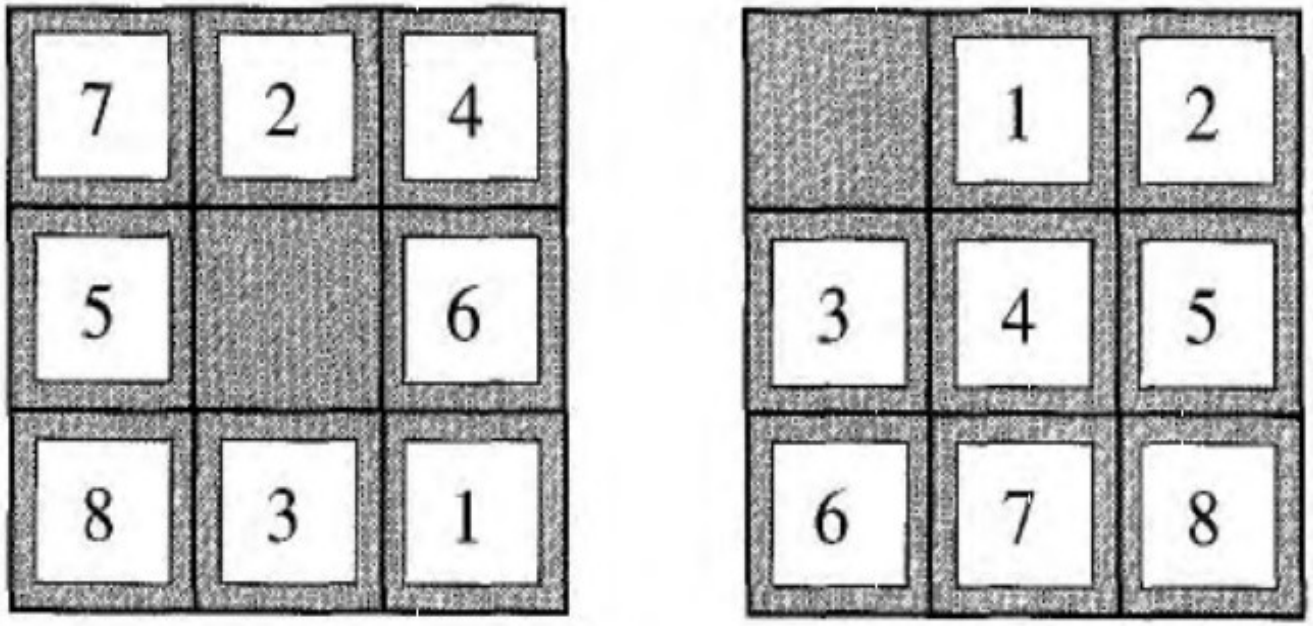
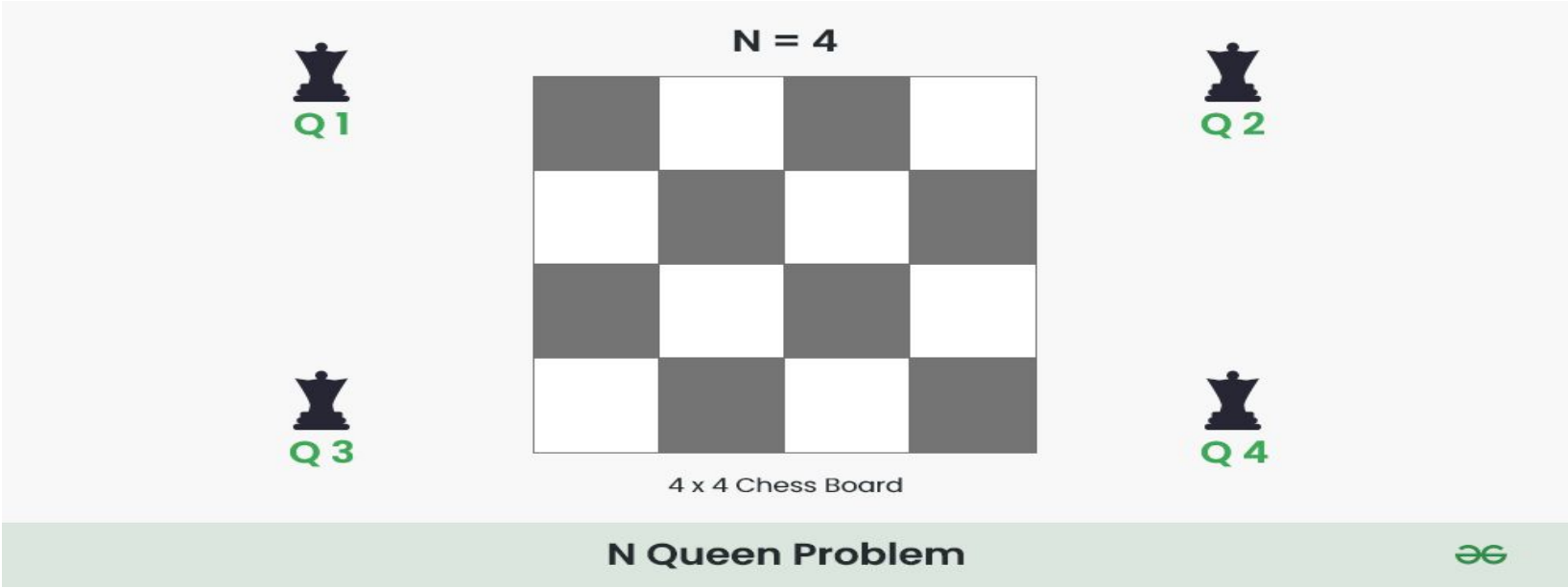


Figure 3.4 A typical instance of the 8-puzzle.

THE 4-QUEENS PROBLEM

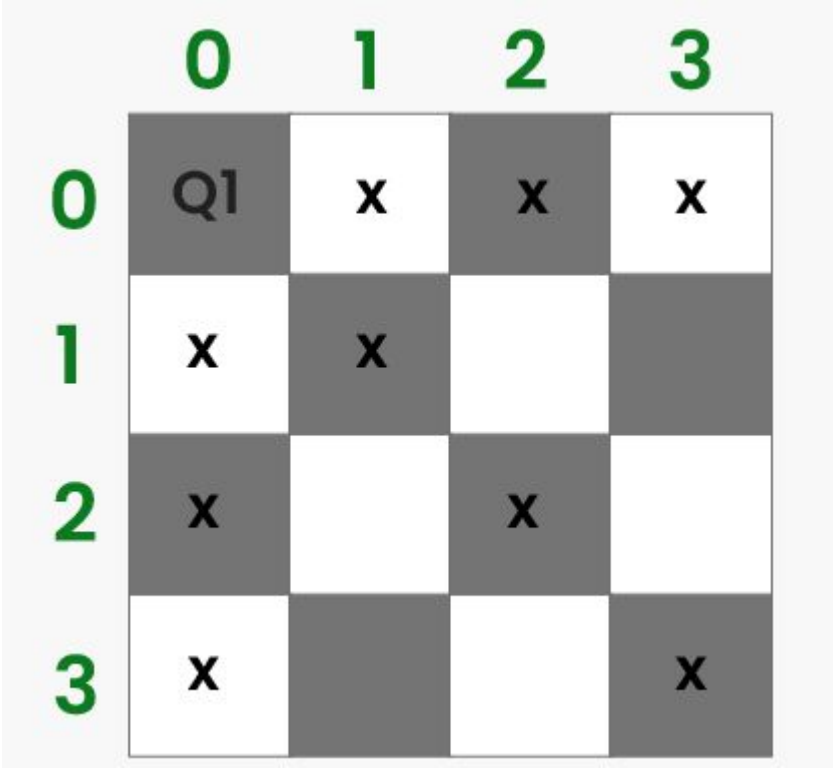
- 4 Queens Problem consists in placing four queens on a **4 x 4 chessboard** so that no two queens attack each other.
- That is, no two queens are allowed to be placed on the same row, the same column or the same diagonal.
- We are going to look for the solution for n=4 on a 4 x 4 chessboard



THE 4-QUEENS PROBLEM

Step 1:

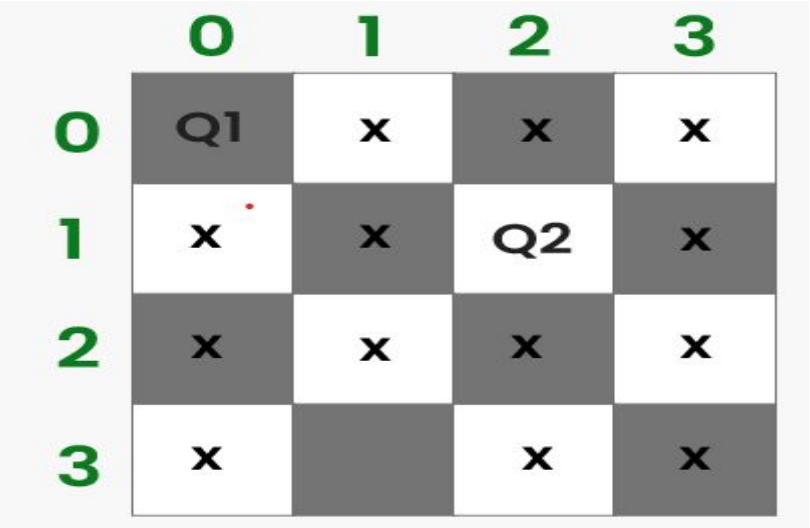
- Put our first Queen (Q1) in the (0,0) cell .
- ‘x‘ represents the cells which is not safe i.e. they are under attack by the Queen (Q1).
- After this move to the next row [0 -> 1].



THE 4-QUEENS PROBLEM

Step 2:

- Put our next Queen (**Q2**) in the **(1,2)** cell .
- After this move to the next row [1 -> 2].



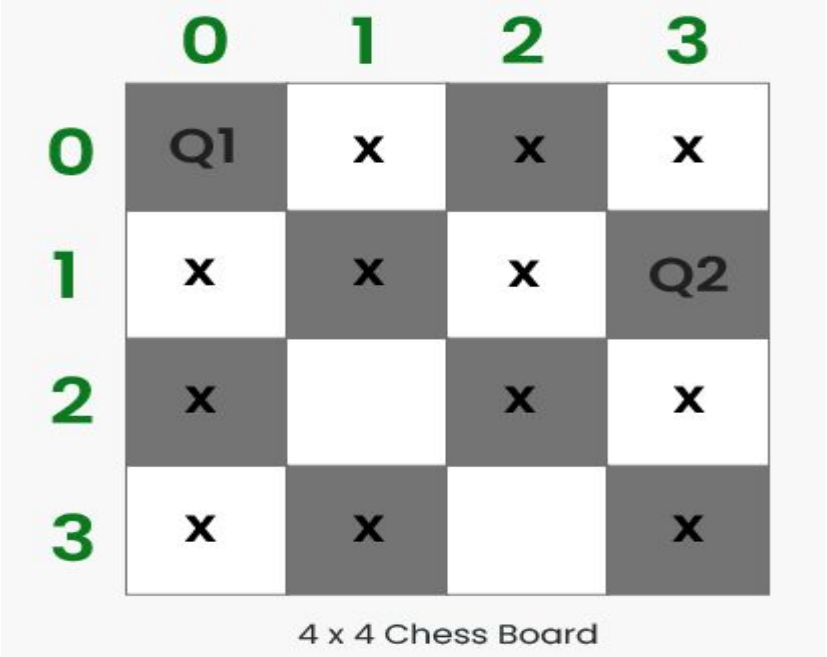
Step 3:

- At row 2 there is no cell which are safe to place Queen (**Q3**) .
- So, backtrack and remove queen **Q2** queen from cell (1, 2) .

THE 4-QUEENS PROBLEM

Step 4:

- There is still a safe cell in the row 1 i.e. cell (1, 3).
- Put Queen (**Q2**) at cell (1, 3).



THE 4-QUEENS PROBLEM

Step 5:

- Put queen **(Q3)** at cell (2, 1).

	0	1	2	3
0	Q1	x	x	x
1	x	x	x	Q2
2	x	Q3	x	x
3	x	x	x	x

THE 4-QUEENS PROBLEM

Step 6:

- There is no any cell to place Queen (Q4) at row 3.

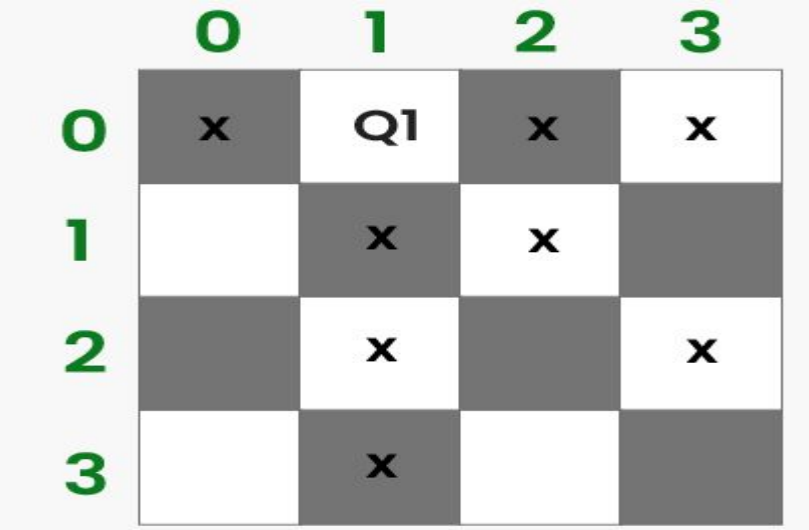
**** Backtrack and remove Queen (Q3) from row 2.

****Again there is no other safe cell in row 2, So backtrack again and remove queen (Q2) from row 1.

****Queen (Q1) will be remove from cell (0,0) and move to next safe cell i.e. (0 , 1).

Step 7:

•Place Queen Q1 at cell (0 , 1), and move to next row.



THE 4-QUEENS PROBLEM

Step 6:

- There is no any cell to place Queen (Q4) at row 3.

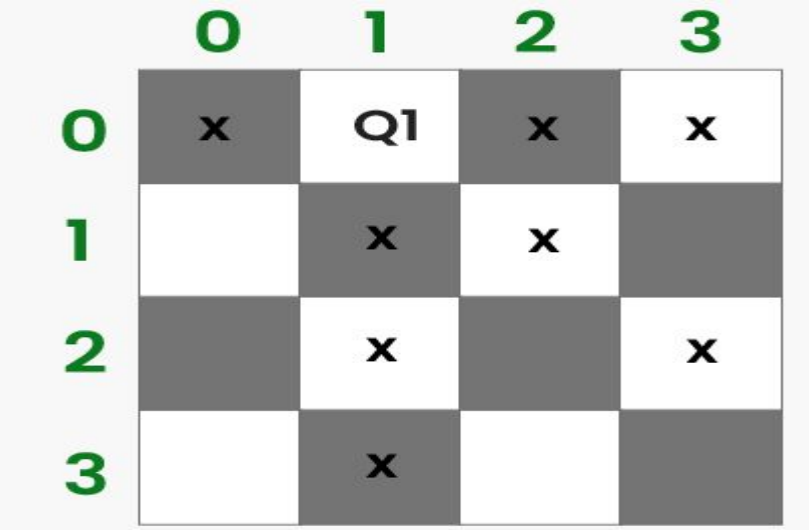
**** Backtrack and remove Queen (Q3) from row 2.

****Again there is no other safe cell in row 2, So backtrack again and remove queen (Q2) from row 1.

****Queen (Q1) will be remove from cell (0,0) and move to next safe cell i.e. (0 , 1).

Step 7:

•Place Queen Q1 at cell (0 , 1), and move to next row.



THE 4-QUEENS PROBLEM

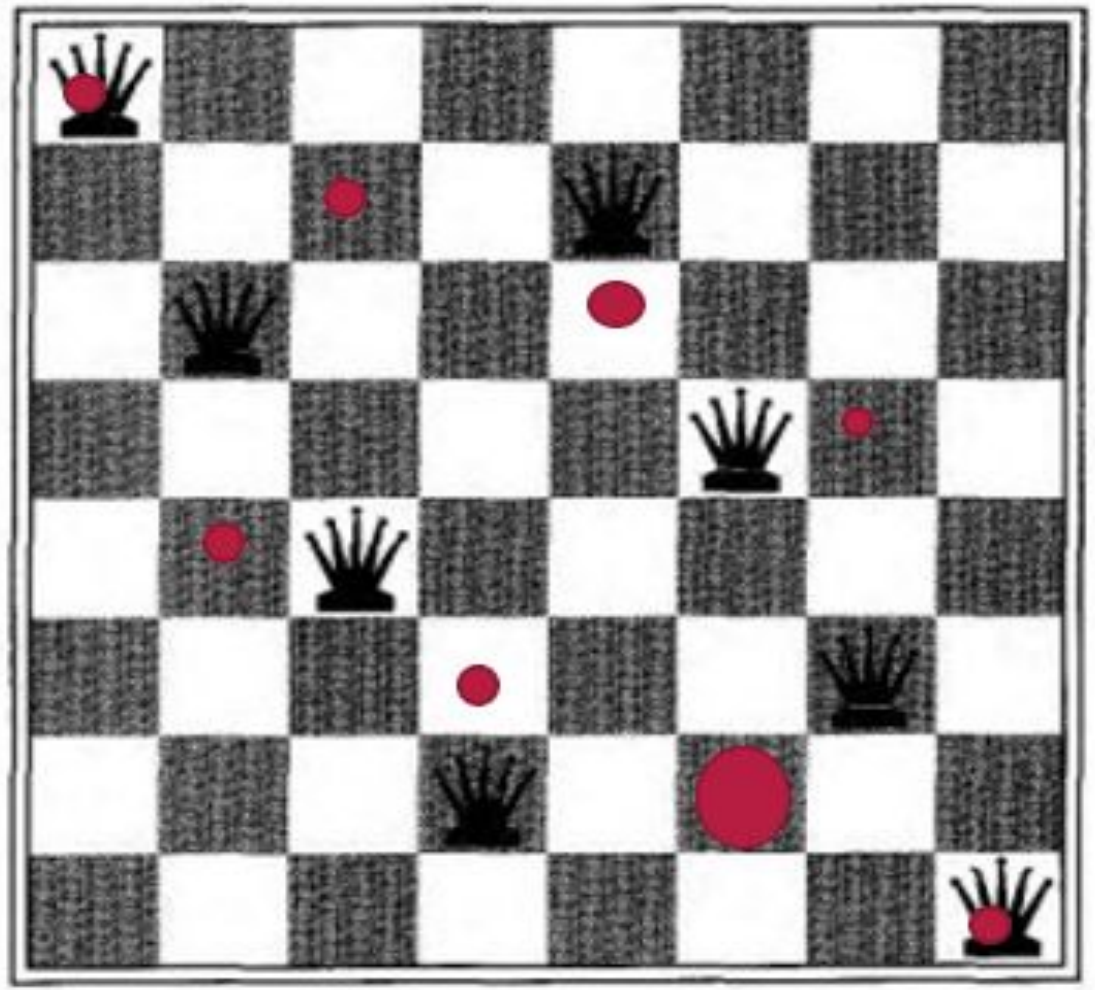
……..

Step 10:

	0	1	2	3
0	x	Q1	x	x
1	x	x	x	Q2
2	Q3	x	x	x
3	x	x	Q4	x

THE 8-QUEENS PROBLEM

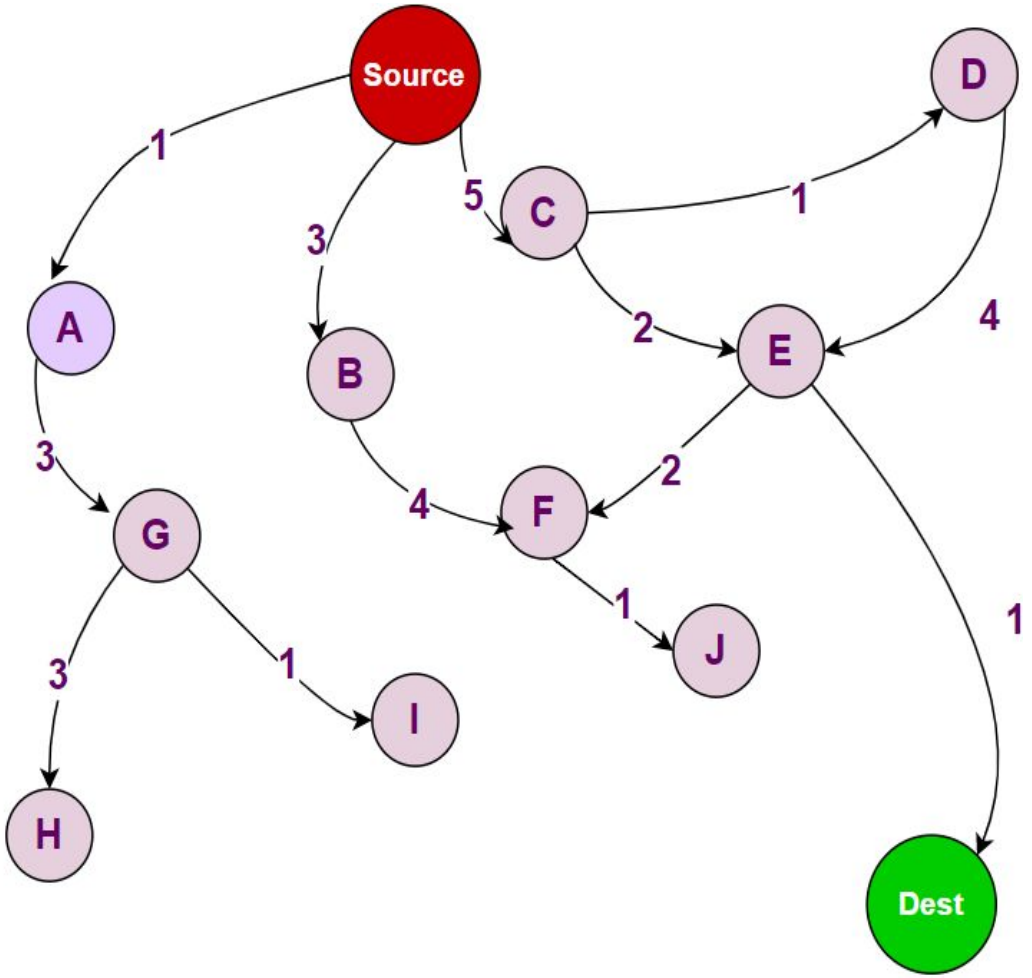
- To place eight queens on a chessboard such that no queen attacks any other.
- There are two main kinds of formulation.
- An incremental formulation involves operators that augment the state description, starting with an empty state; for the 8-queens problem, this means that each action adds a queen to the state.
- A complete-state formulation starts with all 8 queens on the board and moves them around. In either case, the path cost is of no interest because only the final state counts.



UNIFORM-COST SEARCH

- Algorithm for uniform cost search:
- Insert the root node into the priority queue
- Repeat while the queue is not empty:
- Remove the element with the highest priority
- If the removed node is the destination, print **total cost** and stop the algorithm
- Else, enqueue all the children of the current node to the priority queue, with their cumulative cost from the root as priority

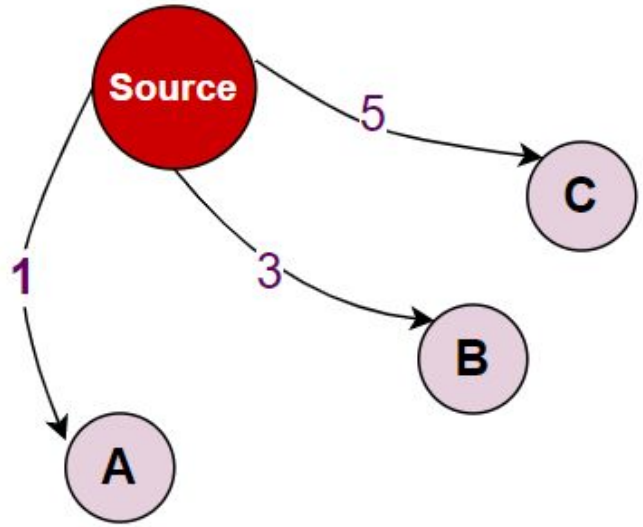
UNIFORM-COST SEARCH

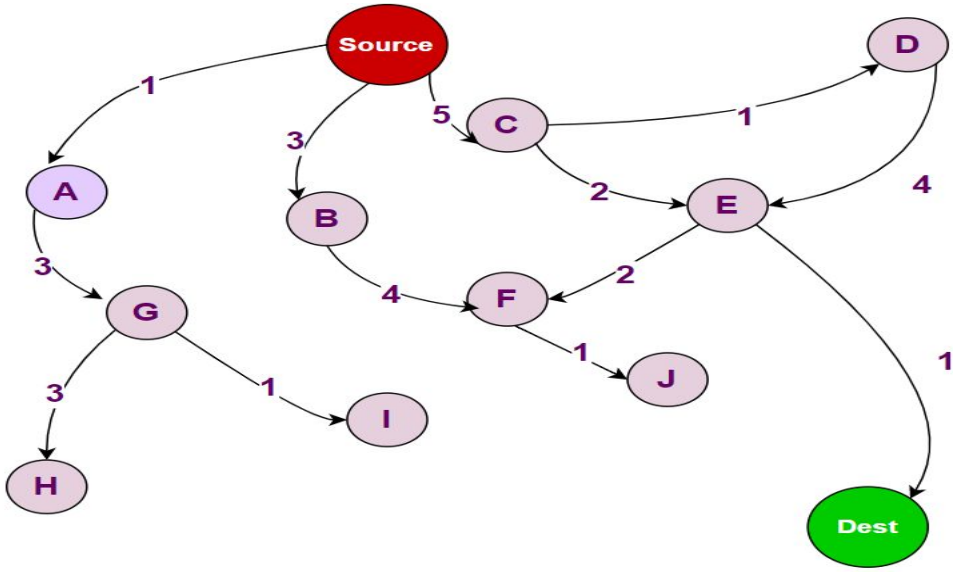


UNIFORM-COST SEARCH

First, we just have the source node in the queue:

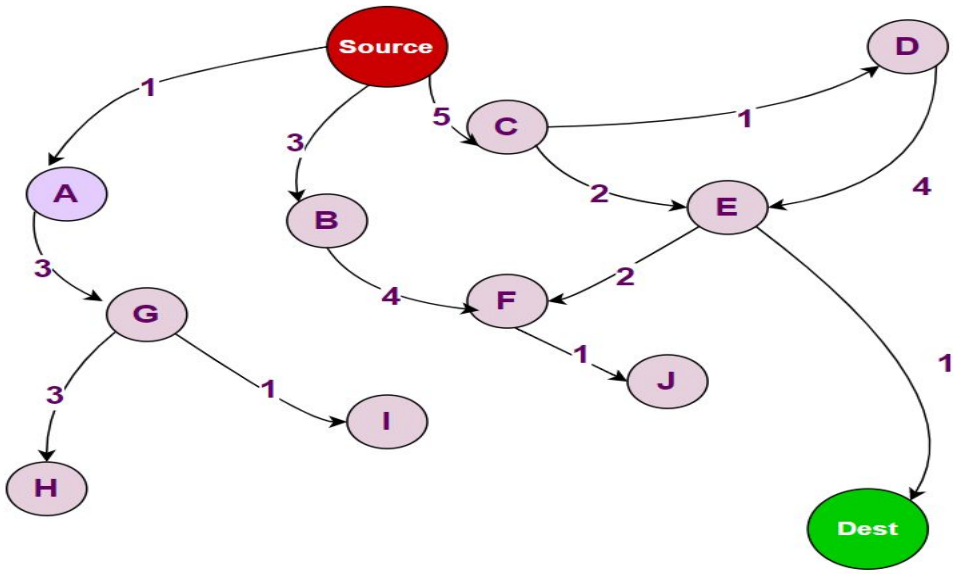
Then, we add its children to the priority queue with their cumulative distance as priority:

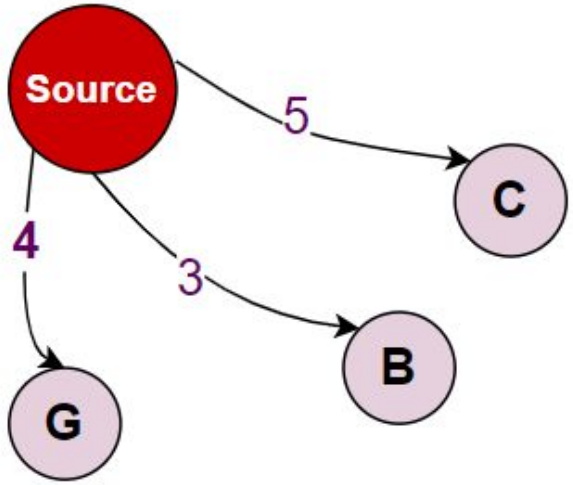




UNIFORM-COST SEARCH

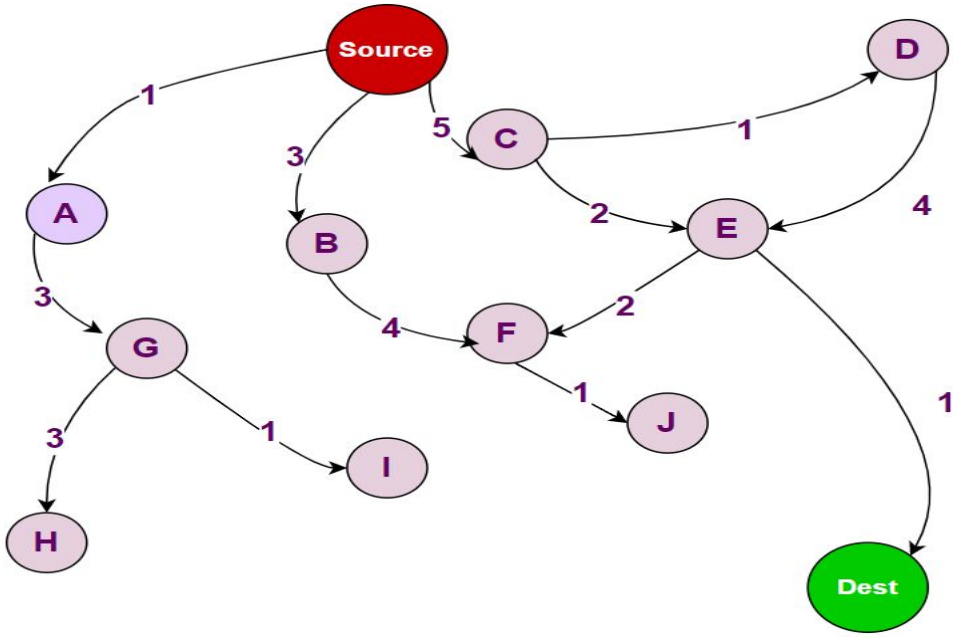
Now, A has the minimum distance (i.e., maximum priority), so it is extracted from the list. Since A is not the destination, its children are added to the PQ.

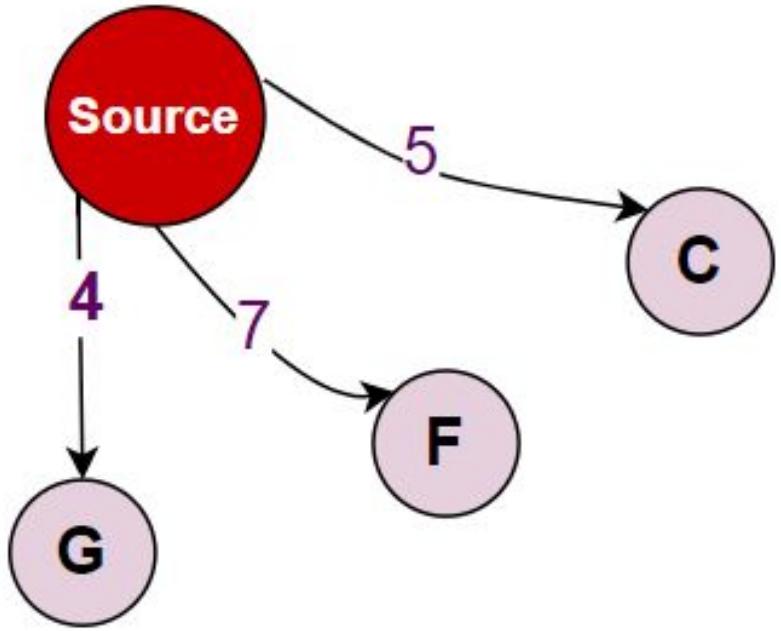




UNIFORM-COST SEARCH

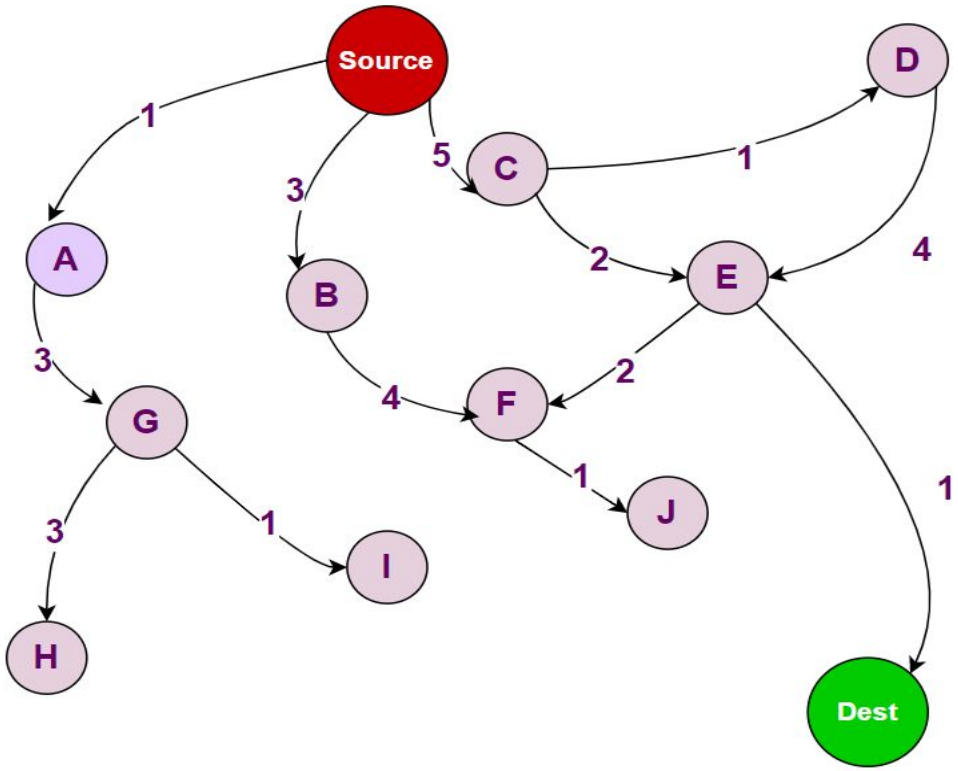
B has the maximum priority now, so its children are added to the queue:

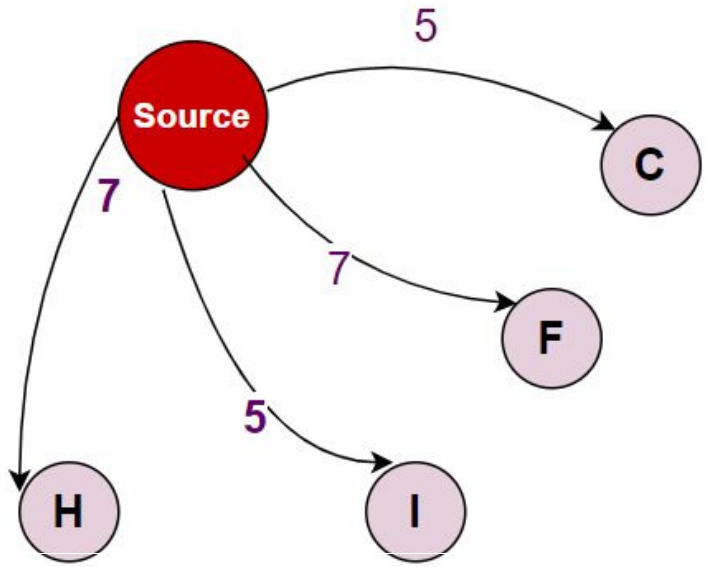




UNIFORM-COST SEARCH

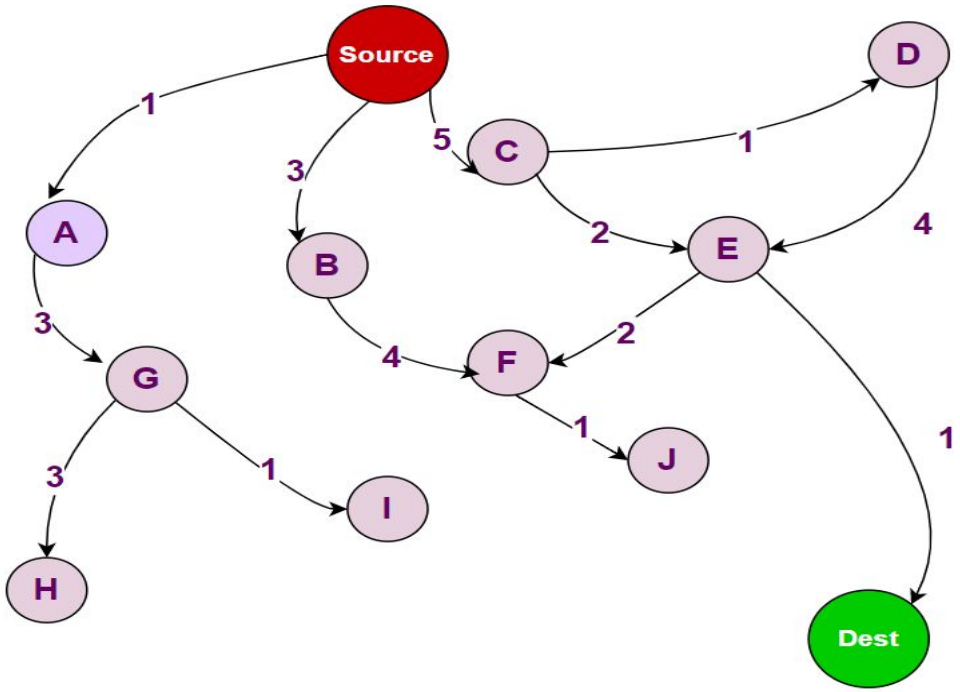
Up next, G will be removed and its children will be added to the queue:

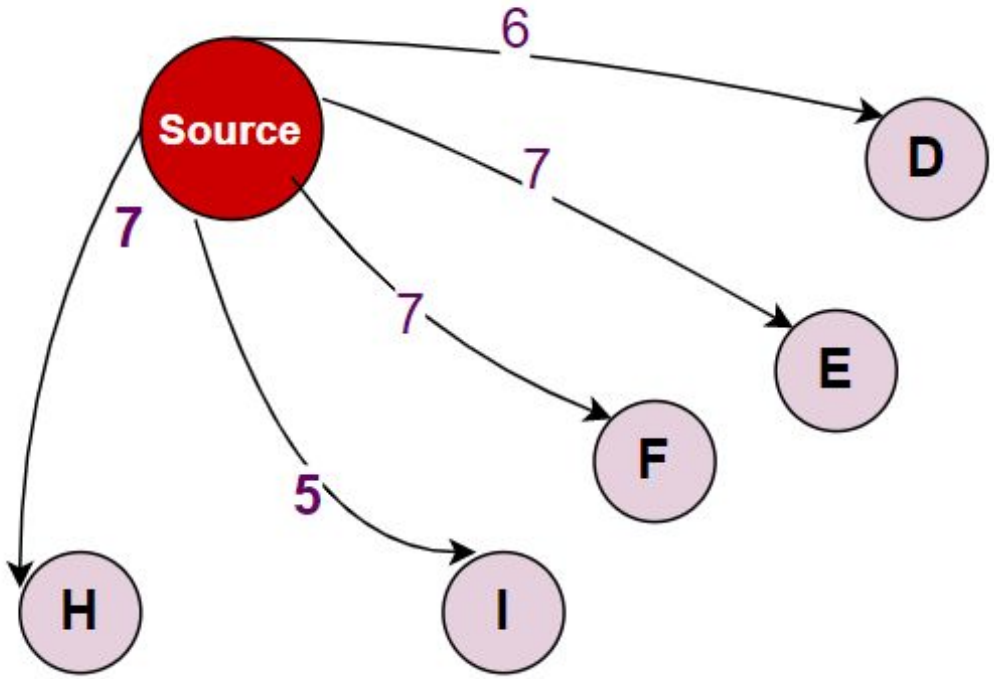




UNIFORM-COST SEARCH

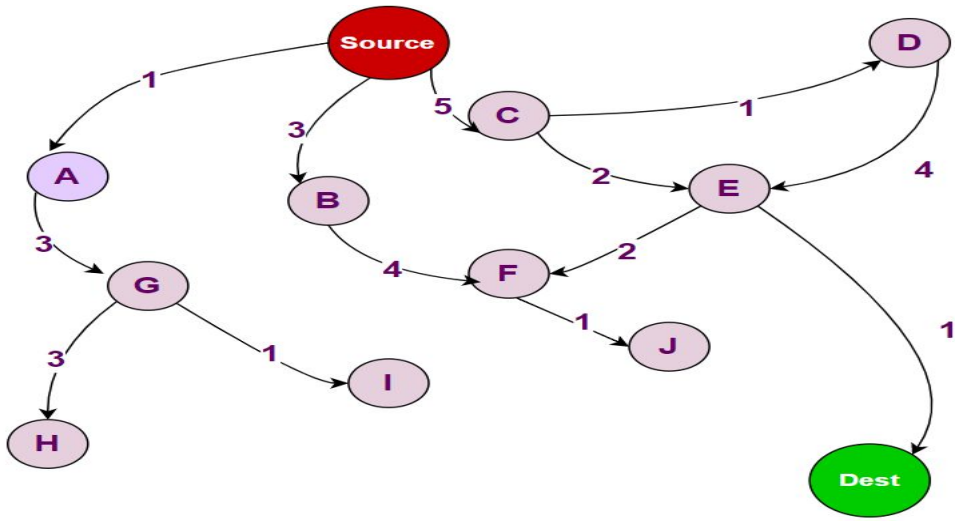
C and I have the same distance, so we will remove alphabetically:

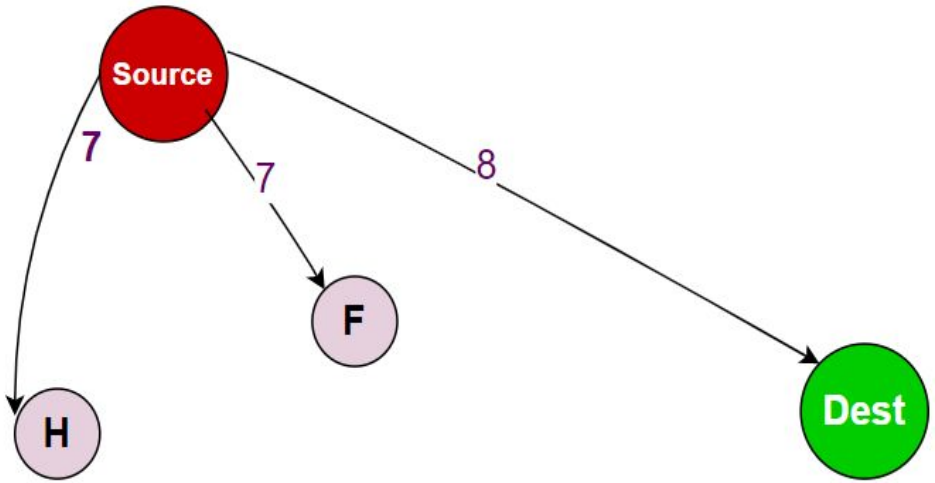




UNIFORM-COST SEARCH

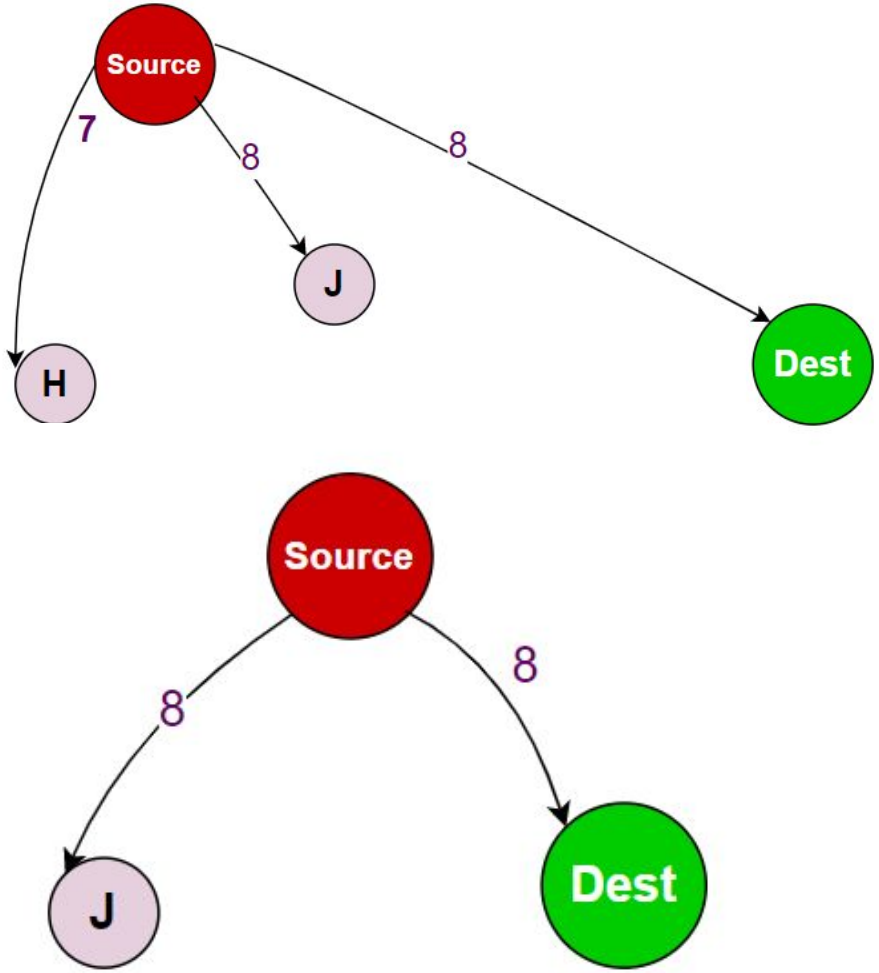
Next, we remove I; however, I has no further children, so there is no update to the queue.
After that, we remove D. D only has one child, E, with a cumulative distance of 10. However, E already exists in our queue with a lesser distance, so we will not add it again.
The next minimum distance is that of E, so that is what we will remove:



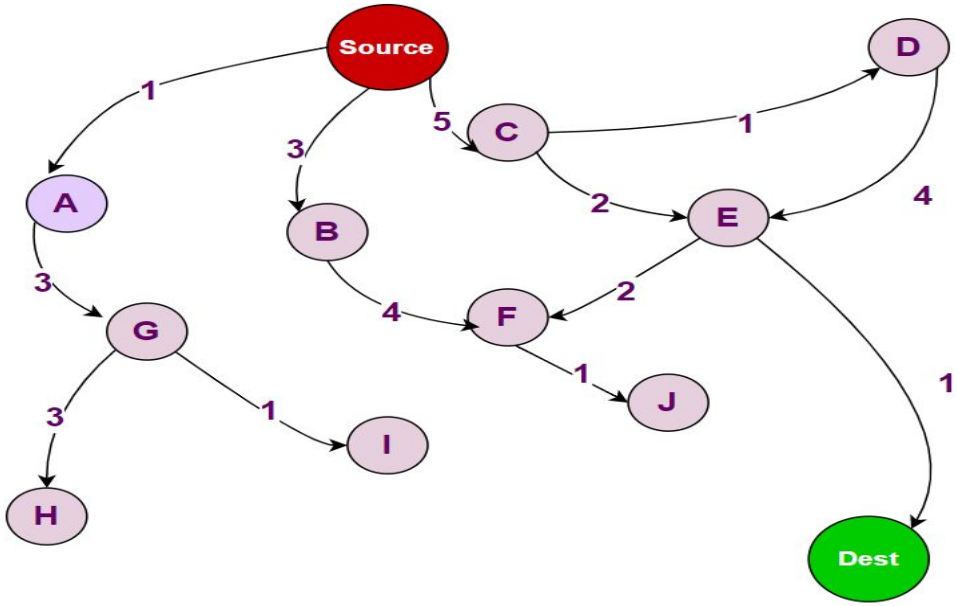


UNIFORM-COST SEARCH

Now, the minimum cost is F, so it will be removed and its child (J) will be added:



After this, H has the minimum cost so it will be removed, but it has no children to be added:



UNIFORM-COST SEARCH

Finally, we remove the Destination node, check that it is our target, and stop the algorithm here.

The minimum distance between the source and destination nodes (i.e., 8) has been found.

Depth First Search

- Depth First Search is an algorithm that explores a tree or graph by starting at the **root node** and **exploring as far as possible** along each branch before backtracking.

- It follows a path from the root to a leaf node, then backtracks to explore other paths.

- This method can be **inefficient** when dealing with large or infinite trees, as it **may** explore deep **branches that do not contain the goal, leading to** wasted time and resources.

DEPTH LIMITED SEARCH

- Depth Limited Search is a modified version of DFS that **imposes** a limit on the depth of the search.

- This means that the algorithm will only explore nodes up to a certain depth, e**ffectively** preventing it from going down excessively deep paths that are unlikely to lead to the goal.

- By setting a **maximum depth limit**, DLS aims to improve efficiency and ensure more manageable search times.

DEPTH LIMITED SEARCH

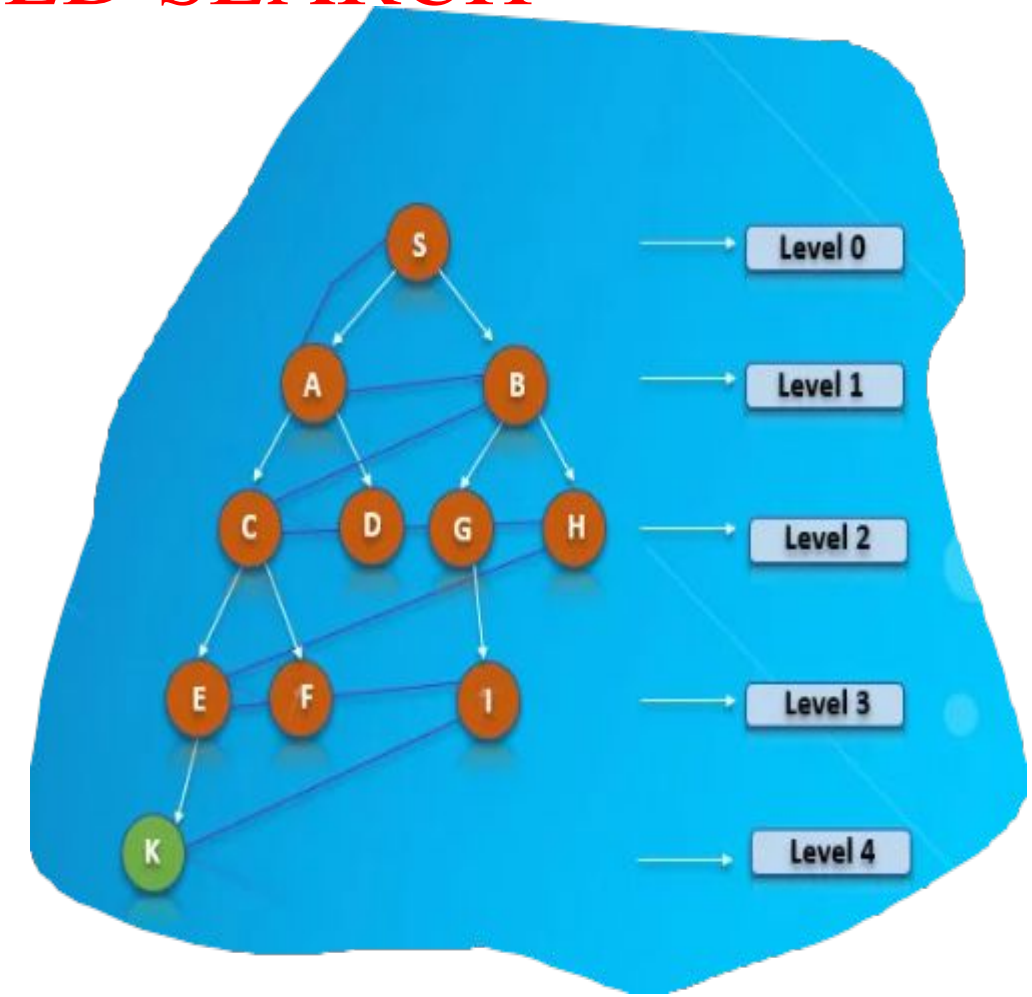
How Depth Limited Search Works

1. **Initialization**: Begin at the root node with a specified depth limit.
2. **Exploration**: Traverse the tree or graph, exploring each node's children.
3. **Depth Check**: If the current depth exceeds the set limit, **stop exploring** that path and backtrack.
4. **Goal Check**: If the goal node is found within the depth limit, the search is successful.
5. **Backtracking**: If the **search reaches the depth limit** or a leaf node without finding the goal, backtrack and explore other branches.

DEPTH LIMITED SEARCH

Applications:

1. Pathfinding in Robotics
2. Network Routing Algorithms
3. Puzzle Solving in AI Systems
4. Game Playing

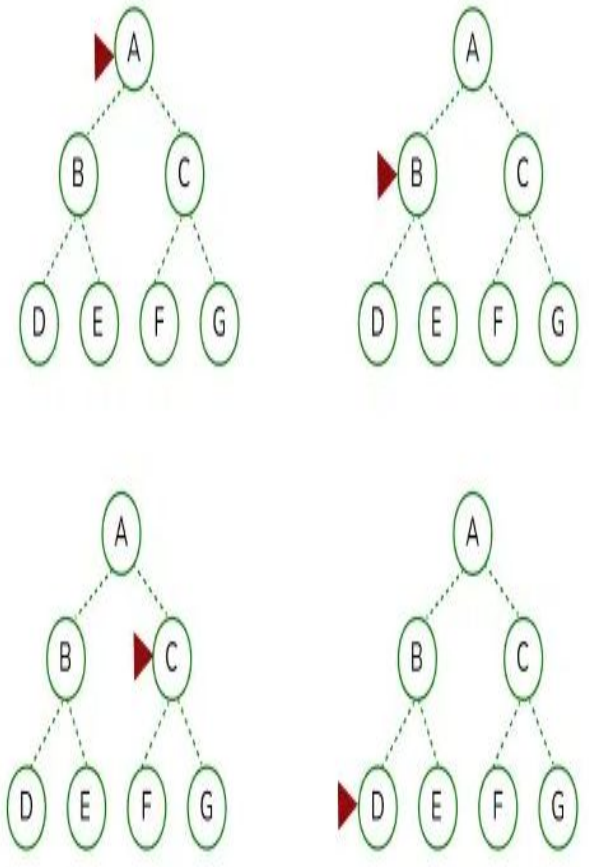


Breadth First Search

- The Breadth-First Search is a traversing algorithm used to satisfy a given property by **searching the tree** or **graph** data structure.
- It belongs to uninformed or blind search AI algorithms as It operates solely **based on the connectivity of nodes** and doesn't prioritize any **particular path** over, another based on heuristic knowledge or domain-specific information.
- it **doesn't incorporate any additional information** beyond the structure of the search space.
- It is optimal for unweighted graphs and is particularly suitable when all actions have the same cost. Due to its systematic search strategy, **BFS can efficiently explore** even infinite state spaces.

Breadth First Search

- Originally it starts at the root node, then it expands all of its successors, it systematically explores all its neighboring nodes before moving to the next level of nodes.
- This process of extending the root node's immediate neighbours, then to their neighbours, and so on, **lasts** until all the nodes within the graph have been visited or until the specific condition is met
- From the above image we can observe that after visiting the node B it moves to node C.
- when the level 1 is completed, it further moves to the next level i.e 2 and **explore node D**. it will move systematically to **node E**, node F and node G. After visiting the **node G it will terminate**.



How BFS Works ??

Context Graph Representation:

- The **city** is represented as a **graph**.
- **Intersections (or decision points** like stop signs or traffic lights) are nodes.
- Roads connecting intersections are edges.

- **Start Node**: Your current location is the start node.

- **Target Node**: Your destination is the target node.
- **Queue Initialization**:
  - The GPS starts at your current location and marks it as visited.
  - It adds all directly connected intersections (roads from your current location) to a queue. accordingly.

Breadth First Search-Example

Exploring Neighboring Nodes:

- The GPS checks each intersection connected to your current location.
- It considers all possible roads you can take and adds them to the queue if they haven't been visited.

Shortest Path Discovery:

- As BFS continues, it explores all possible routes level by level (all roads of the same distance).
- Once it reaches the target intersection (your destination), BFS identifies the shortest path.

Path Reconstruction: The GPS reconstructs the shortest route based on the nodes visited and provides directions accordingly.

Breadth First Search-Example

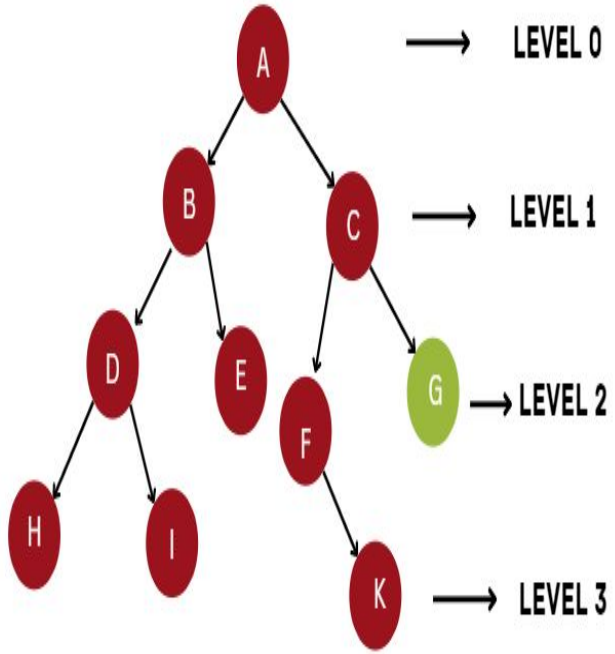
How **GPS navigation systems** find the shortest route between two locations on a map ??

BFS in GPS Navigation Scenario:

- Imagine you're using a GPS to navigate from your current location to a destination in a city.
- The city map consists of **roads (edges)** and intersections (nodes).
- The goal of the GPS is to **find the shortest path** from your starting point to your destination.

Iterative Deepening Search

- Iterative Deepening Search (IDS) is a search strategy that combines the benefits of depth-first search (DFS) and breadth-first search (BFS).
- It works by repeatedly applying depth-limited depth-first search (DFS) with increasing depth limits until a goal is found.
- This method ensures that the search is both complete (like BFS) and uses minimal memory (like DFS).



Iterative Deepening Search

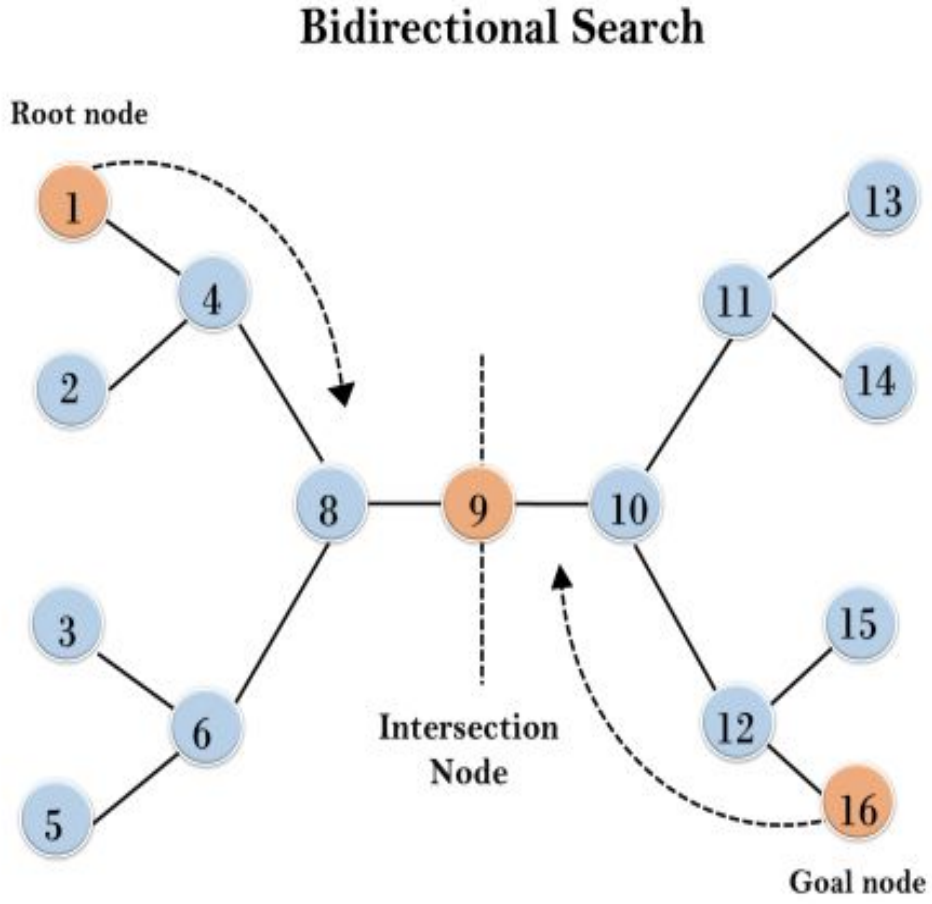
How Iterative Deepening Search Works

- **Initialization**: Start with a depth limit of 0.
- **Depth-limited DFS**: Perform a depth-limited DFS up to the current depth limit.
- **Increase Depth:** If the goal is not found, increment the depth limit by 1.
- **Repeat**: Repeat the depth-limited DFS with the new depth limit.
- **Termination:** Stop when the goal is found or all nodes have been searched..

BIDIRECTIONAL SEARCH

- Bidirectional Search is an **efficient search algorithm in AI** that works by simultaneously searching forward from the initial state and backward from the goal state.

- The **search stops** when the two searches meet, **significantly reducing the number of nodes explored** compared to traditional search algorithms.



BIDIRECTIONAL SEARCH

How Bidirectional Search Works

1. **Initialization**: Begin with two frontiers: one from the initial state (forward search) and one from the goal state (backward search).
2. **Search**: Expand nodes alternately from the front and back until the two frontiers intersect.
3. **Termination**: The search terminates when a common node is found in both frontiers, indicating that the forward and backward paths have met.
4. **Path Construction**: Combine the paths from the initial state to the intersection node and from the goal state to the intersection node to form the complete path.

BIDIRECTIONAL SEARCH

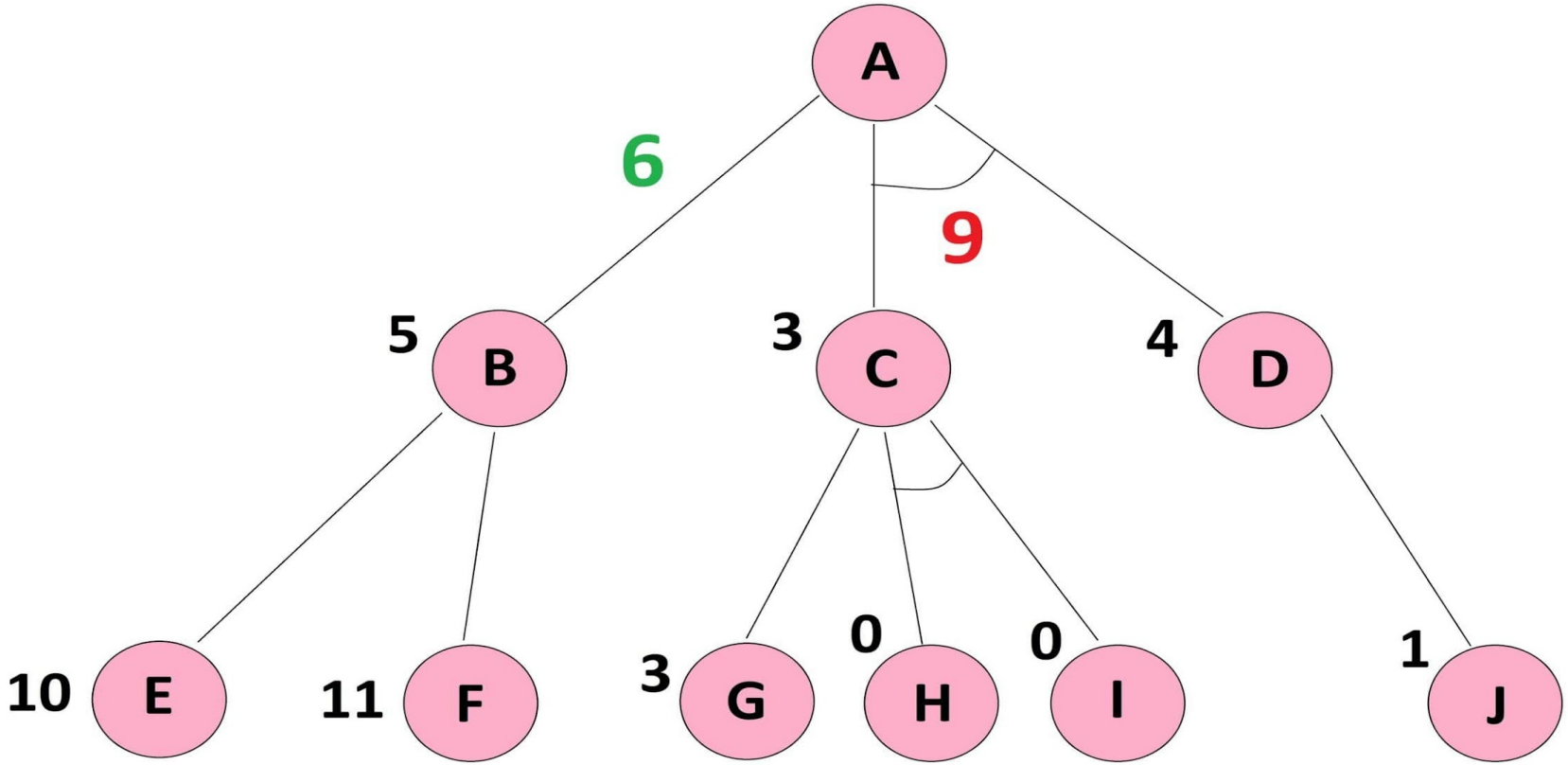
Advantages:

- Bidirectional search is fast.
- Bidirectional search requires less memory

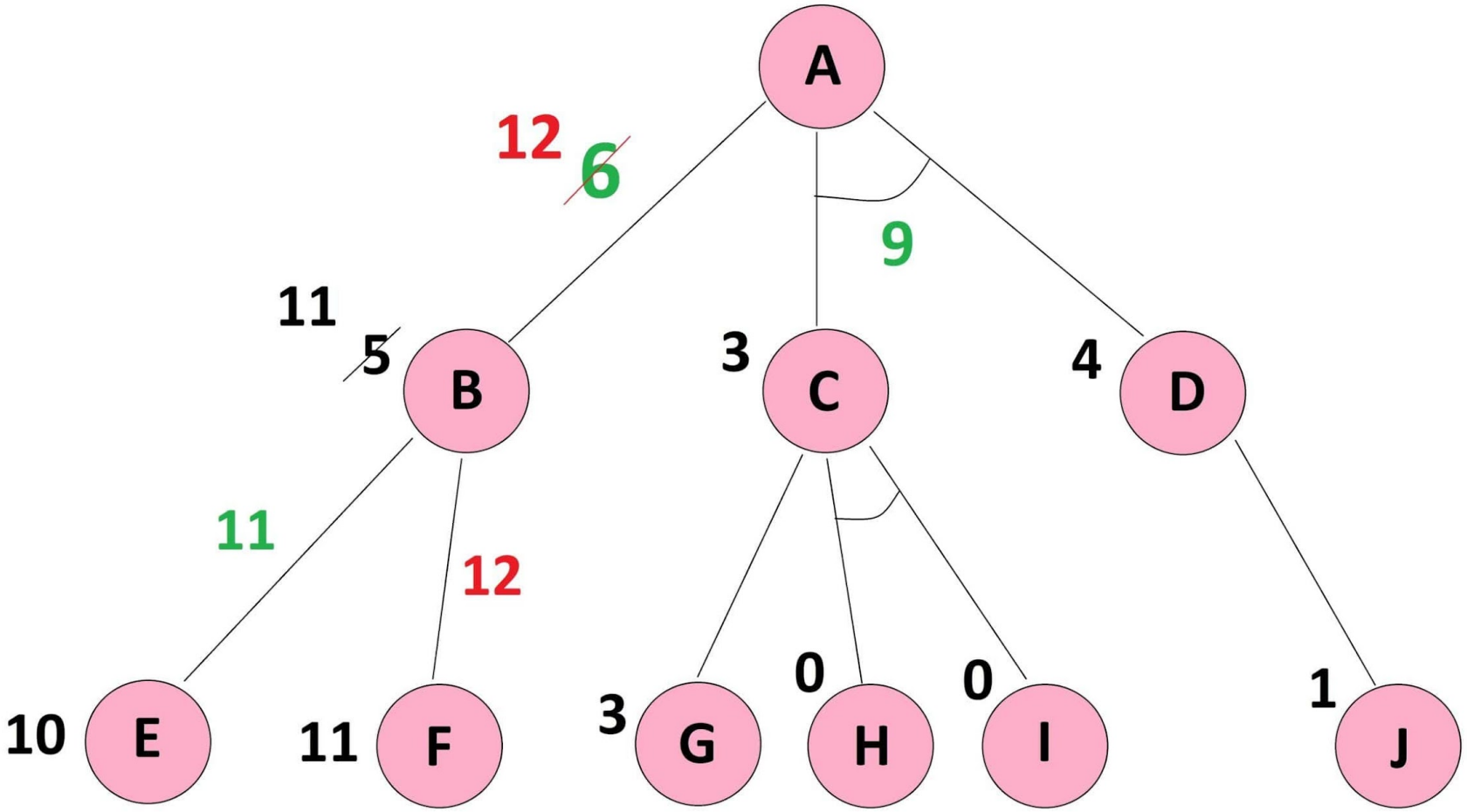
Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- **In bidirectional search, one should know the goal state in advance.**

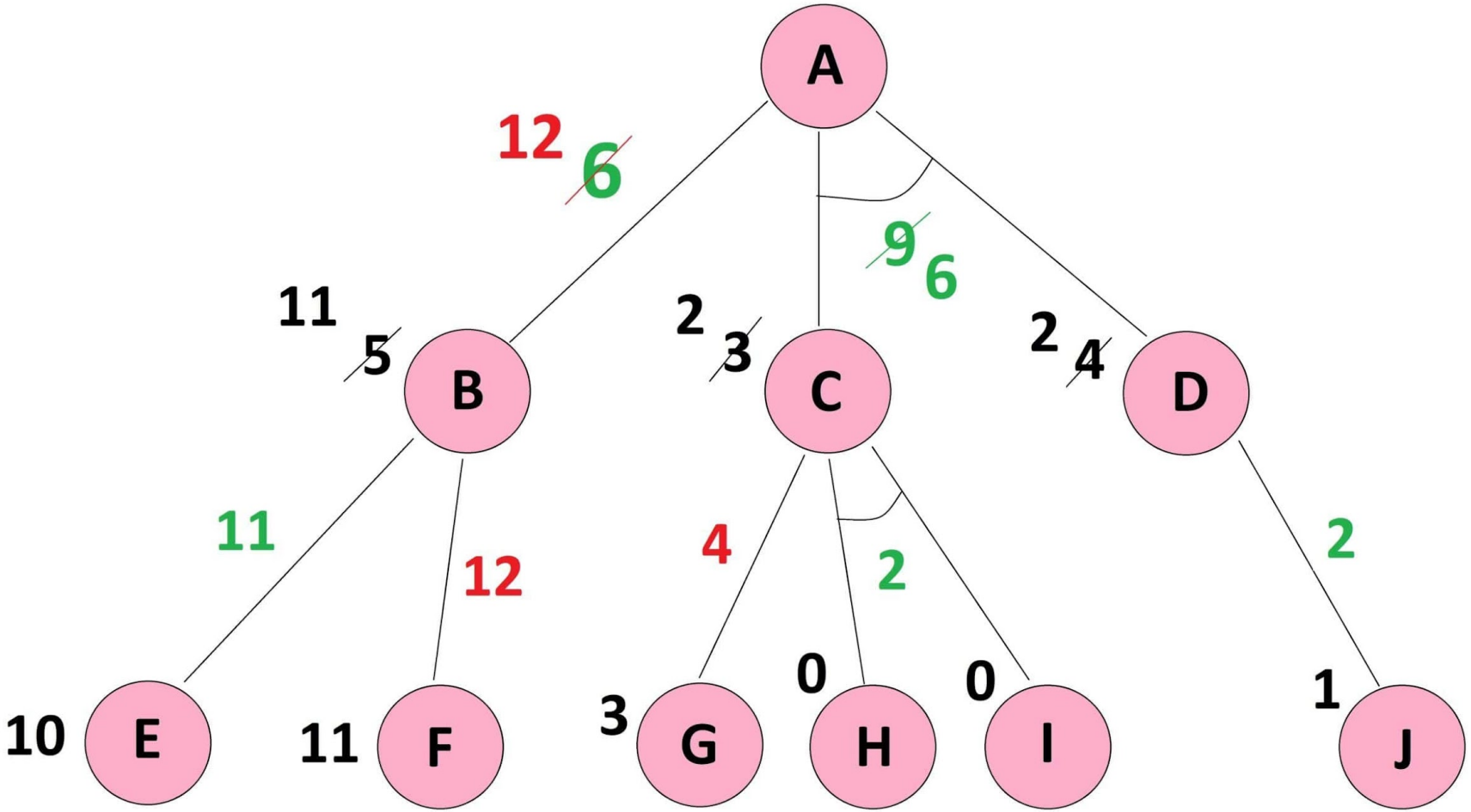
Forward Propagation



Reaching the Last Level and Back Propagation



Correcting the Path from Start Node



Real-Life Applications of AO* algorithm:

Vehicle Routing Problem:

- The vehicle routing problem is determining the shortest routes for a fleet of vehicles to visit a set of customers and return to the depot, while minimizing the total distance travelled and the total time taken. The AO* algorithm can be used to find the optimal routes that satisfy both objectives.

Portfolio Optimization:

- Portfolio optimization is choosing a set of investments that maximize returns while minimizing risks. The AO* algorithm can be used to find the optimal portfolio that satisfies both objectives, such as maximizing the expected return and minimizing the standard deviation.

Subject Name:-Foundations of Artificial Intelligence Systems

Subject Code:- 21BTCS016

Unit No.:- 2

Lecture No.:- 6

Topic Name :- Adversarial Search

Dr. Arvind Jagtap
Department of CSE

Informed Search Algorithms

- Informed search algorithm contains an array of knowledge such as how far we are from **the goal, path cost, how to reach to goal node,** etc.
- This knowledge help agents to explore less to the search space and find more efficiently the goal node.
- The informed search algorithm is **more useful for large search space**. Informed search algorithm uses the **idea of heuristic**, so it is also called Heuristic search.
- **Heuristics function:** Heuristic is a function which is used in Informed Search, and it finds the **most promising path**.
- It takes the current state of the agent as its input and produces the **estimation of how close agent is from the goal.**
- The heuristic method, however, might **not always give the best solution, but it guaranteed to find a good solution in reasonable time**.
- Heuristic function estimates how close a state is to the goal.
- It is represented by **h(n), and it calculates the cost of an optimal path between the pair of states.**
- The value of the **heuristic function is always positive.**

Admissibility of the heuristic function

- h(n) <= h*(n)
- **Here h(n) is heuristic cost, and h*(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.**

Pure Heuristic Search:

- Pure heuristic search is the simplest form of heuristic search algorithms.
- It expands nodes based on their heuristic value h(n).
- It maintains two lists, **OPEN and CLOSED list**. In the **CLOSED list, it places those nodes which have already expanded** and in **the OPEN list, it places nodes which have yet not been expanded.**
- On each iteration, each **node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list**.
- The algorithm continues unit a goal state is found.

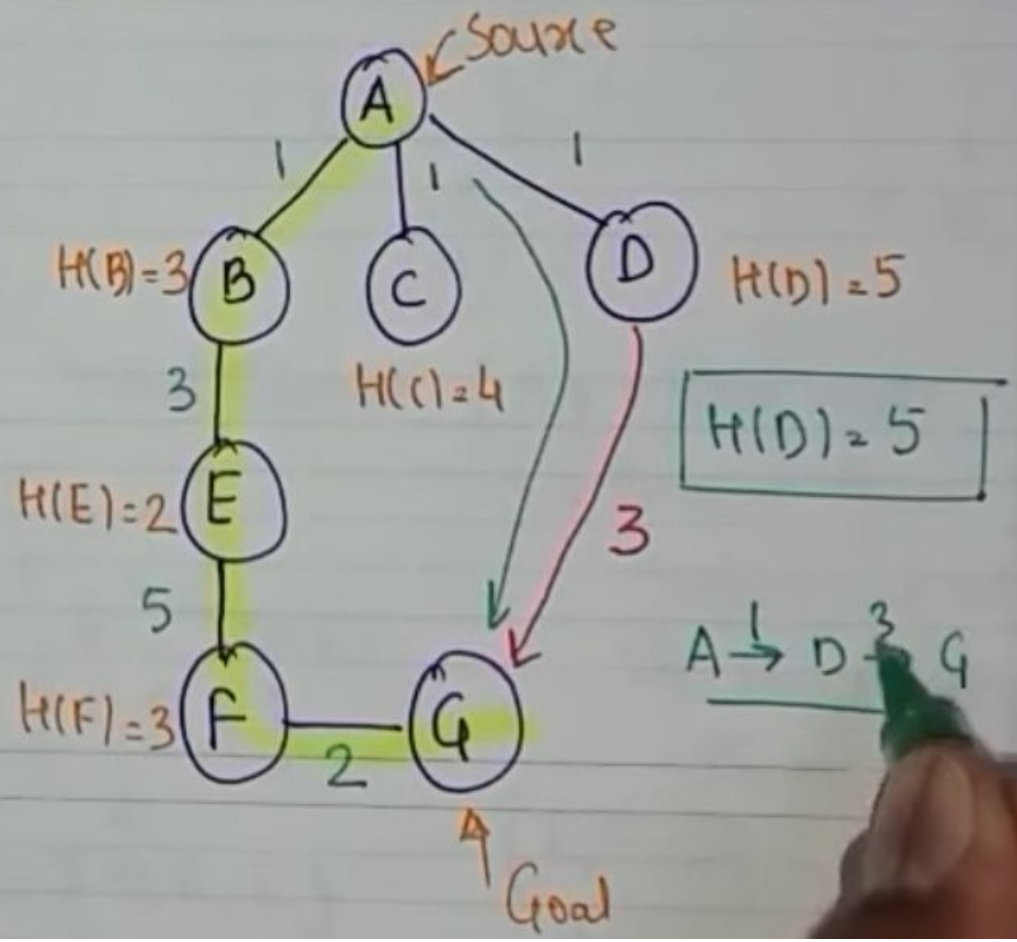
of Heuristic:- (undersestimate)

(i) Admissible: In this Heuristic function, ver overestimates the cost of reaching ne Goal. $H(n) \leq H'(n)$ {Goal}

↳ h(n) is always less than or equal to actual cost of lowest-cost path from node 'n' to goal.

(ii) Non-Admissible :- Overestimate

$H(n) > H'(n)$



$H(B)=3$, $H(C)=4$, $H(D)=5$

$f(n) = H(n) + G(n)$

$B=4$, $C=5$, $D=6$.

Actual = (11)

$3 < 11$

1.Best-first Search Algorithm (Greedy Search):

- Greedy best-first search algorithm always selects the path which appears best at that moment.
- It is the combination of depth-first search and breadth-first search algorithms.
- It uses the heuristic function and search.
- Best-first search allows us to take the advantages of both algorithms.
- With the help of best-first search, at each step, we can choose the most promising node.
- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function.

i.e. $f(n)= h(n)$. Were, $h(n)$= estimated cost from node n to the goal.

- The greedy best first algorithm is implemented by the priority queue.

Best first search algorithm

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.
- **Step 4:** Expand the node n, and generate the successors of node n.
- **Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

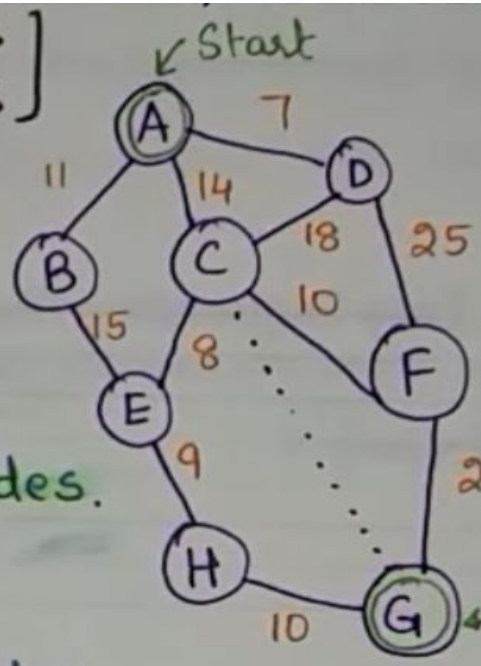
BEST FIRST SEARCH: GREEDY SEARCH. [BFS / DFS]

↳ Uses evaluation Algorithm (func^{n}) to decide which adjacent node is most promising and then explore.

↳ Category of Heuristic or Informed Search.

→ Priority Queue is used to Store Cost of nodes.

↳ Sorted order.



Straight Line distance.

A → G = 40
B → G = 32
C → G = 25
D → G = 35
E → G = 19
F → G = 17
G → G = 0
H → G = 10

ALGORITHM:

Priority Queue 'PQ' Containing initial States

Loop

if PQ = Empty Return FAIL

Else

NODE ← Remove_First(PQ)

if NODE = GOAL

Return Path from Initial to NODE

Else

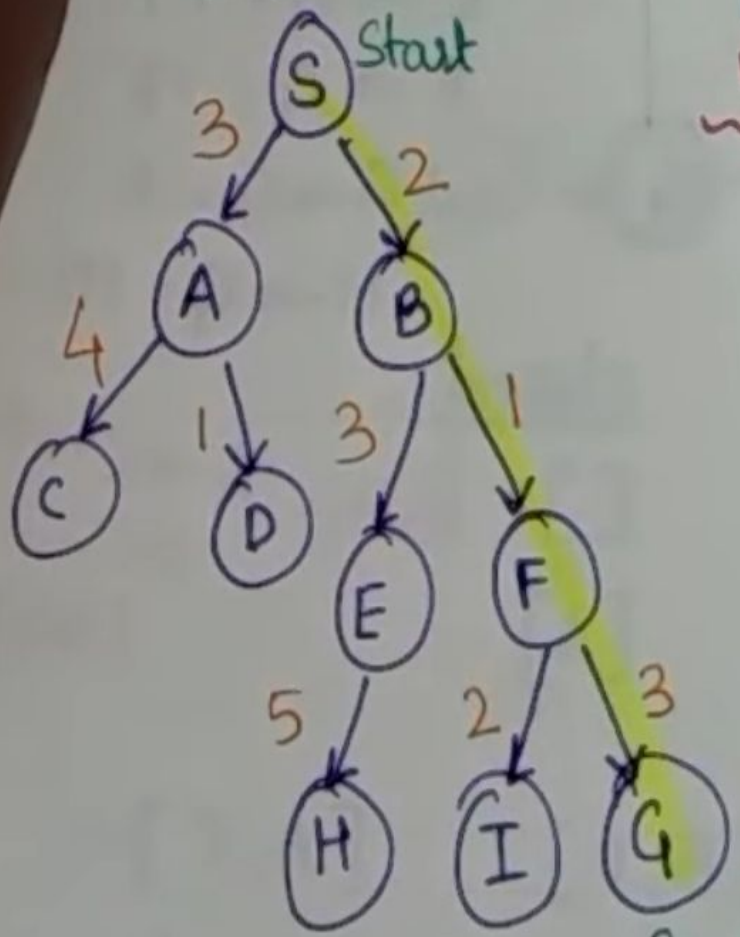
Generate all Successors of NODE and insert newly generated NODE into PQ according to COST Value.

END LOOP

open	closed
[A]	[]
[C, B, D]	[A]
[B, D]	[A, C]
[F, E, B, D]	[A, C]
[E, B, D]	[A, C, F, G]

Path = A → C → F → G

est First Search Example:



node(n)	H(n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Open	Closed
[B, A]	[S]
[F, E, A]	[S, B]
[G, I, E, A]	[S, B, F]
	[S, B, F, G]

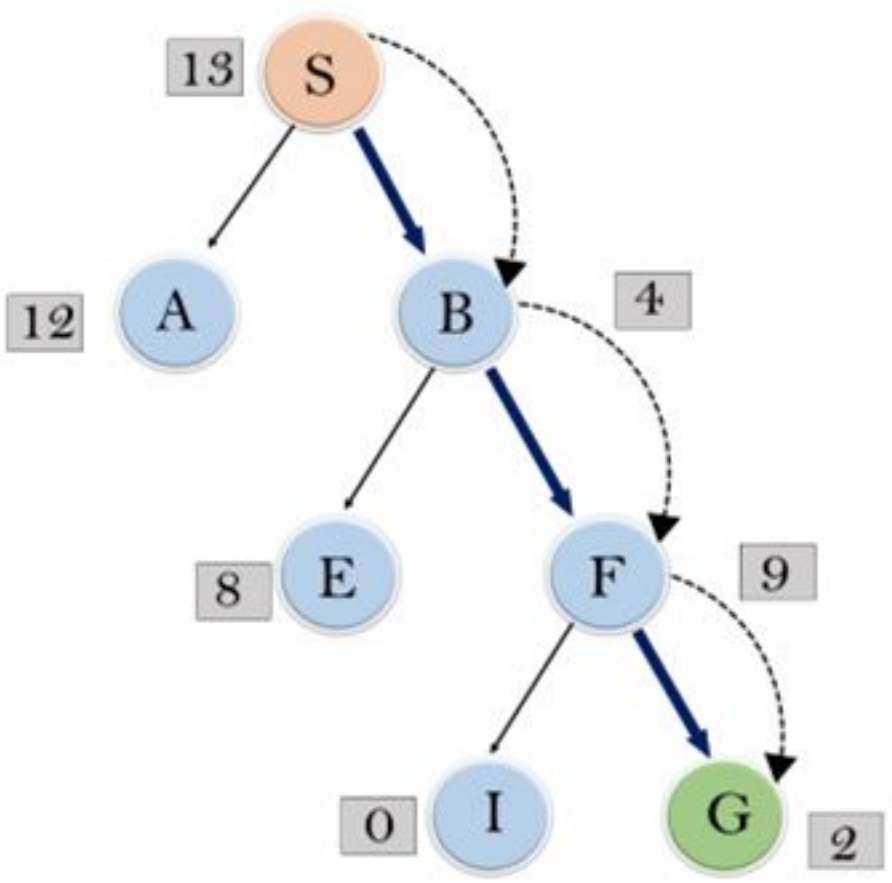
Path = S → B → F → G

Example:

- Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function f(n)=h(n) , which is given in the below table.

•In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists.

- Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list
Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]

: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]

: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path : S----> B----->F----> G

- **Time Complexity:** The worst case time complexity of Greedy best first search is $O(b^m)$.
- **Space Complexity:** The worst case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.
- **Complete:** Greedy best-first search is also incomplete, even if the given state space is finite.

Advantages:

- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

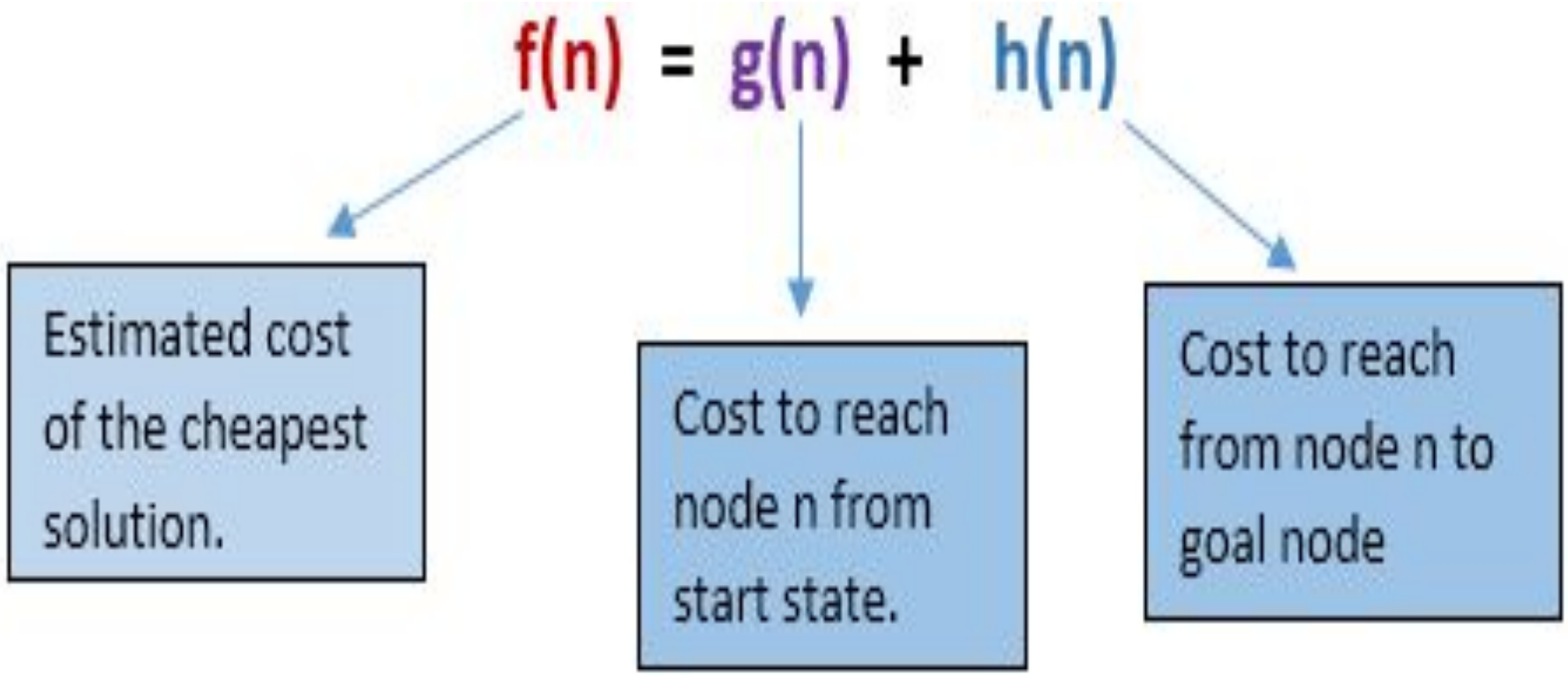
Disadvantages:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

A* Search Algorithm:

- A* search is the most commonly known form of best-first search
- It uses heuristic function h(n), and cost to reach the node n from the start state g(n).
- It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently.
- A* search algorithm finds the shortest path through the search space using the heuristic function.
- This search algorithm expands less search tree and provides optimal result faster. A* algorithm is similar to UCS except that it uses **g(n)+h(n)** instead of g(n).
- In A* search algorithm, we use search heuristic as well as the cost to reach the node.

- We can combine both costs as following, and this sum is called as a **fitness number**.


f(n) = g(n) + h(n)
Estimated cost of the cheapest solution.
Cost to reach node n from start state.
Cost to reach from node n to goal node

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors,and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

Step 6: Return to **Step 2**.

Ques:- Explain A* Algorithm for Search.

↳ Uses heuristic func^n h(n) and cost to reach the node 'n' from start state g(n).

↳ Finds shortest path through search space.

↳ Fast and optimal result.

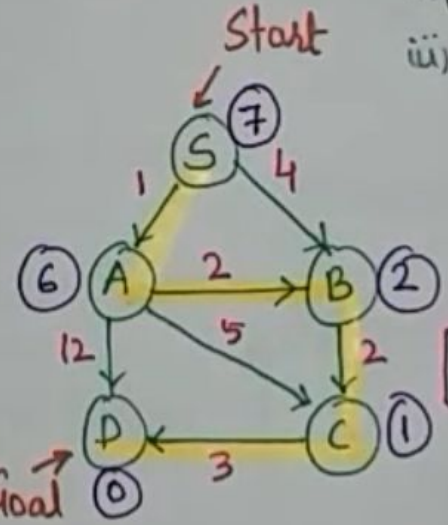
$f(n) = g(n) + h(n)$ ⟶ heuristic Value (child node)

f(n): estimated cost.

g(n): ↳ Cost to reach node

ALGORITHM:

↳ i) Enter starting node in OPEN list.
ii) If OPEN list is empty return FAIL
iii) Select node from OPEN list which has smallest value (g+h).
↳ if node = Goal, return Success
iv) Expand node 'n' and generate all successors
↳ Compute (g+h) for each successor node.
v) if node 'n' is already in OPEN/CLOSED, attach to back pointer.
vi) Go to (ii)



S→A = 1+6 = 7 ✓
S→B = 4+2 = 6 ✓
S→B→C = 4+2+1 = 7 ✓
S→B→C→D = 4+2+3+0 = 9 r1
S→A→B = 1+2+2 = 5 ✓
S→A→C = 1+5+1 = 7 ✓
S→A→D = 1+12 = 13 r2
S→A→B→C = 1+2+2+1 = 6 ✓
(#) S→A→B→C→D = 1+2+2+3 = 8 r3
S→A→C→D = 1+5+3 = 9 r4

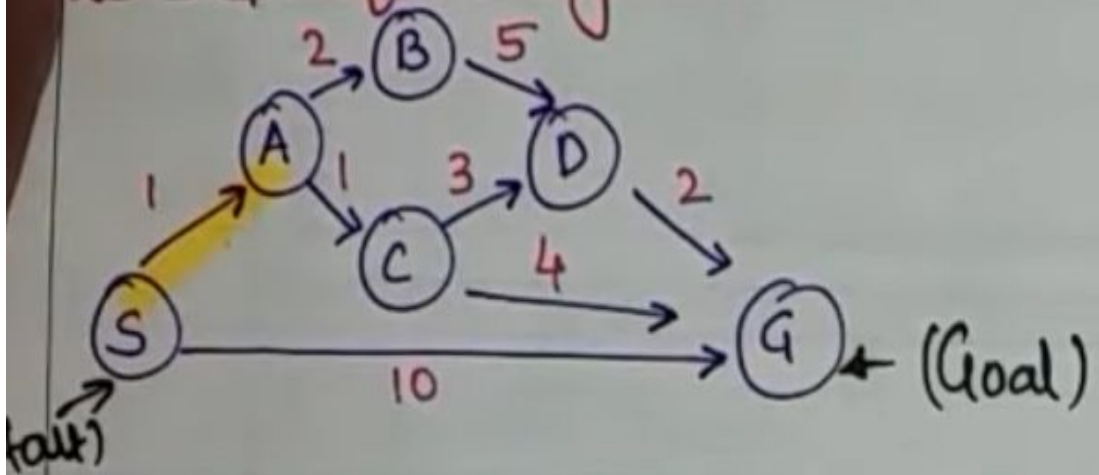
Advantages:-

↳ i) Best searching algorithms.
ii) optimal and complete.
iii) Solving complex problems.

Disadvantages:-

↳ i) Doesn't always produces shortest.
ii) Complexity issues.
iii) Requires memory.

Example of A* Algorithm:-



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

$S \to A = 1+3 = 4$

$S \to G = 10+0 = 10$

$S \to A \to B = 1+2+4 = 7$ ✓

$S \to A \to C = 1+1+2 = 4$ ✓

$S \to A \to C \to D = 1+1+3+6 = 11$ ✓

$= S \to A \to C \to G = 1+1+4 = 6$.

$S \to A \to B \to D = 1+2+5+6 = 14$ =

$S \to A \to C \to D \to G = 1+1+3+2 = 7$

$S \to A \to B \to D \to G = 1+2+5+2 = 10$

Advantages:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

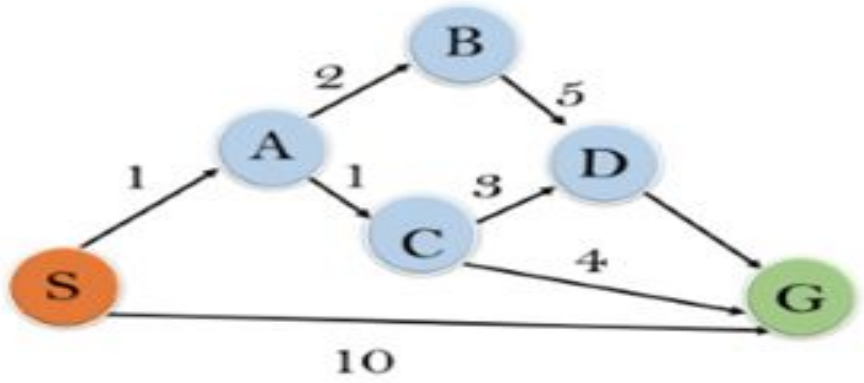
Disadvantages:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Example:

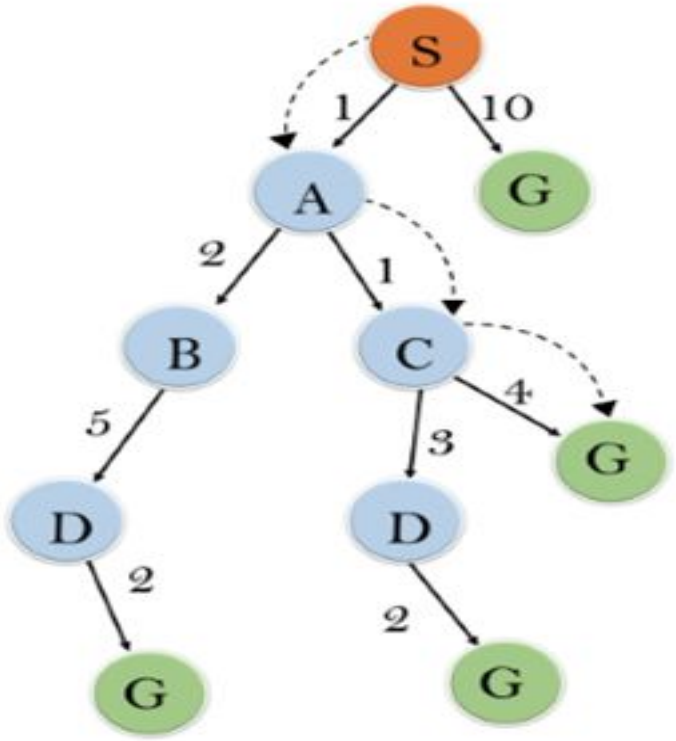
In this example, we will traverse the given graph using the A* algorithm. The heuristic value of all states is given in the below table so we will calculate the f(n) of each state using the formula f(n)= g(n) + h(n), where g(n) is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



State	h(n)
S	5
A	3
B	4
C	2
D	6
G	0

Solution:



Initialization: {(S, 5)}

Iteration1: {(S--> A, 4), (S-->G, 10)}

Iteration2: {(S--> A-->C, 4), (S--> A-->B, 7), (S-->G, 10)}

Iteration3: {(S--> A-->C--->G, 6), (S--> A-->C--->D, 11), (S--> A-->B, 7), (S-->G, 10)}

Iteration 4 will give the final result, as **S--->A--->C--->G** it provides the optimal path with cost 6.

Points to remember:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition f(n)
- **Complete:** A* algorithm is complete as long as:
- Branching factor is finite.
- Cost at every action is fixed.

Optimal: A* search algorithm is optimal if it follows below two conditions:

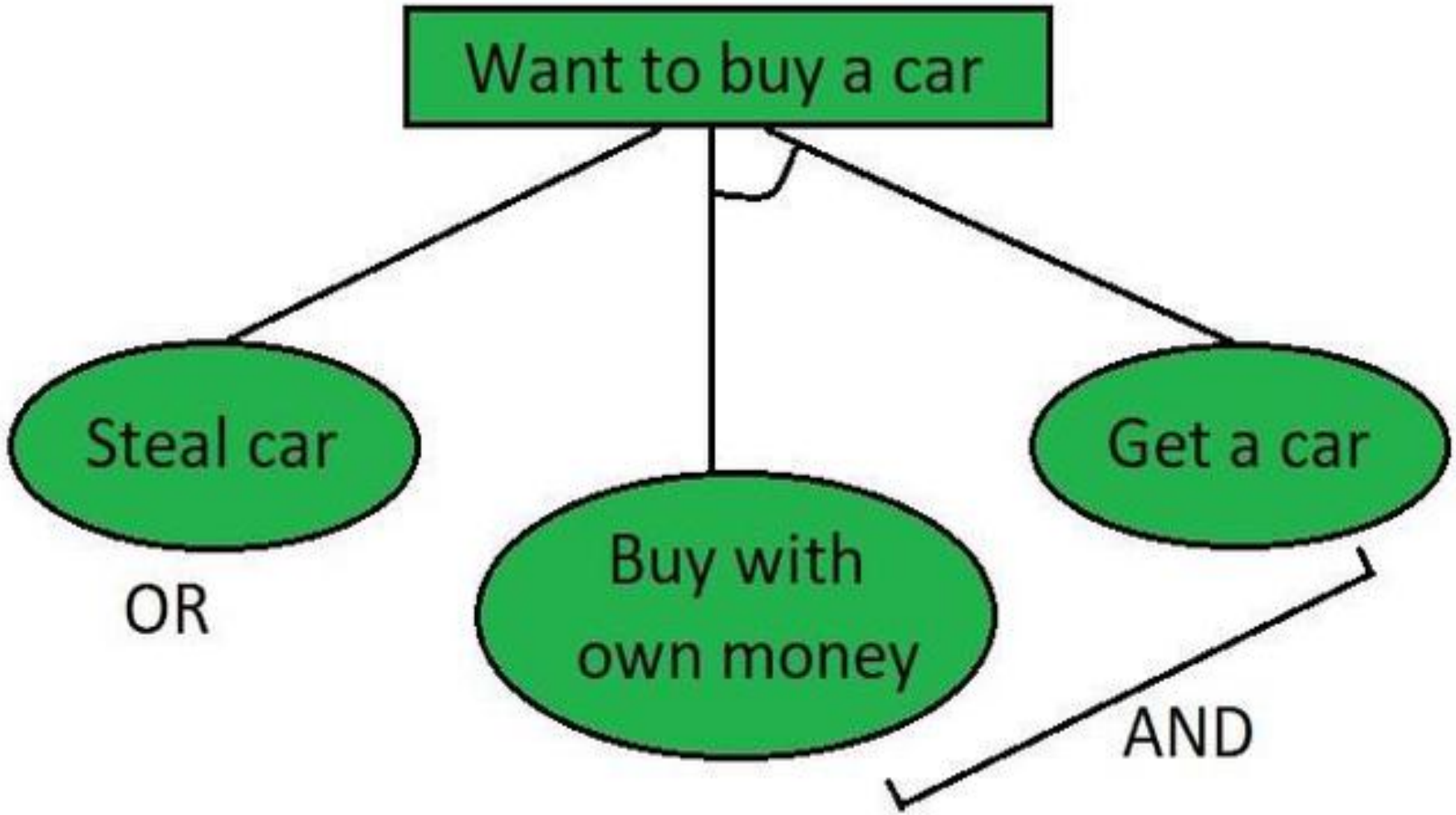
- **Admissible:** the first condition requires for optimality is that h(n) should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

If the heuristic function is admissible, then A* tree search will always find the least cost path.

- **Time Complexity:** The time complexity of A* search algorithm depends on heuristic function, and the number of nodes expanded is exponential to the depth of solution d.
- The time complexity is O(b^d), where b is the branching factor.
- **Space Complexity:** The space complexity of A* search algorithm is **O(b^d)**

AO* Algorithm

- The AO* method **divides** any given difficult **problem into a smaller group** of problems that are then resolved **using the AND-OR** graph concept. AND OR graphs are specialized graphs that are used in problems that can be divided into smaller problems.
- The AND side of the graph represents a set of tasks that must be completed to achieve the main goal, while the OR side of the graph represents different methods for accomplishing the same main goal.


Want to buy a car
Steal car
OR
Buy with own money
Get a car
AND

Working of AO* algorithm:

The evaluation function in AO* looks like this:

f(n) = g(n) + h(n)

f(n) = Actual cost + Estimated cost

here,

f(n) = The actual cost of traversal.

g(n) = the cost from the initial node to the current node.

h(n) = estimated cost from the current node to the goal state.

- Like A* algorithm here we will use two arrays and one heuristic function.
- **OPEN:** It contains the nodes that have been traversed but yet not been marked solvable or unsolvable.
- **CLOSE:** It contains the nodes that have already been processed.
- **h(n):** The distance from the current node to the goal node.

AO* Search Algorithm

Step 1: Place the starting node into OPEN.

Step 2: Compute the most promising solution tree say T0.

Step 3: Select a node n that is both on OPEN and a member of T0. Remove it from OPEN and place it in CLOSE

Step 4: If n is the terminal goal node then levelled n as solved and levelled all the ancestors of n as solved. If the starting node is marked as solved then success and exit.

Step 5: If n is not a solvable node, then mark n as unsolvable. If starting node is marked as unsolvable, then return failure and exit.

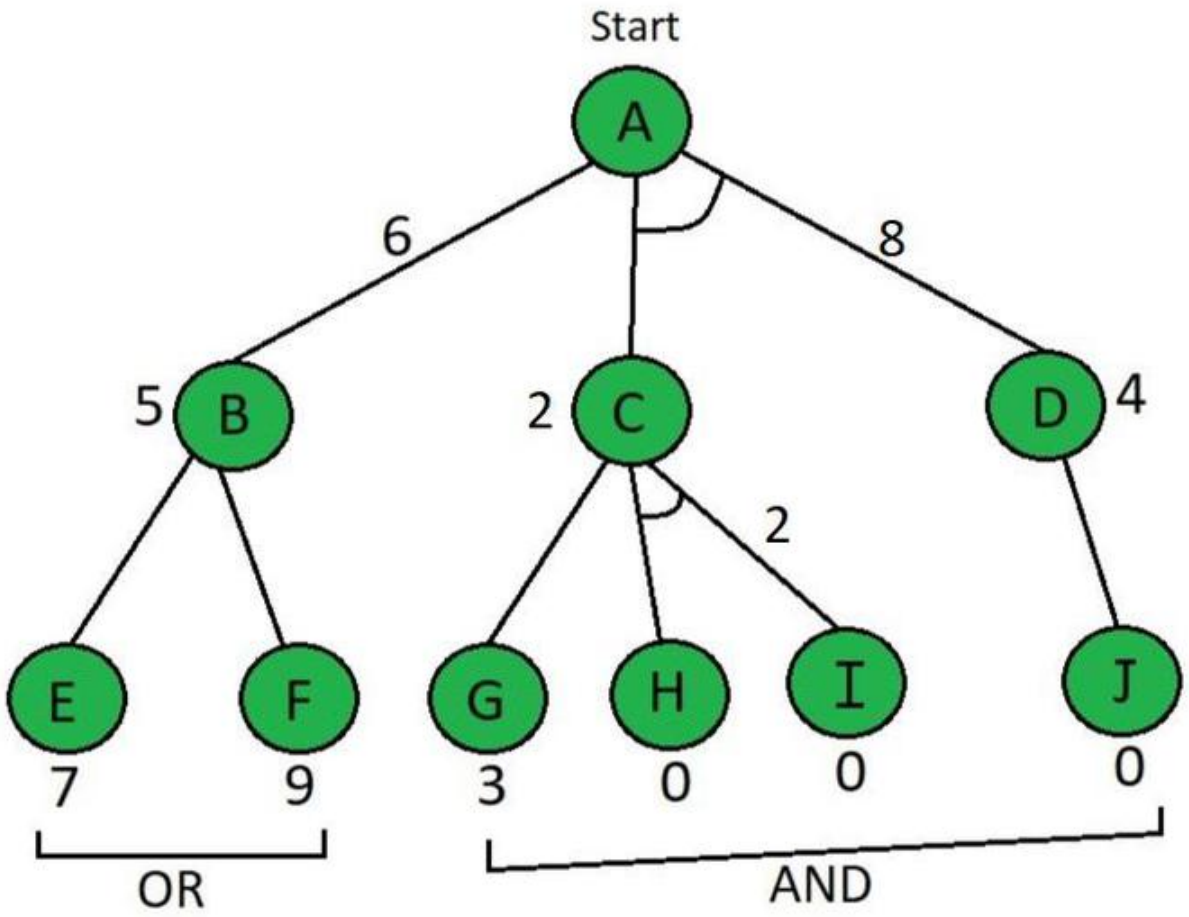
Step 6: Expand n.

Find all its successors and find their h (n) value, push them into OPEN.

Difference between the A* Algorithm and AO* algorithm

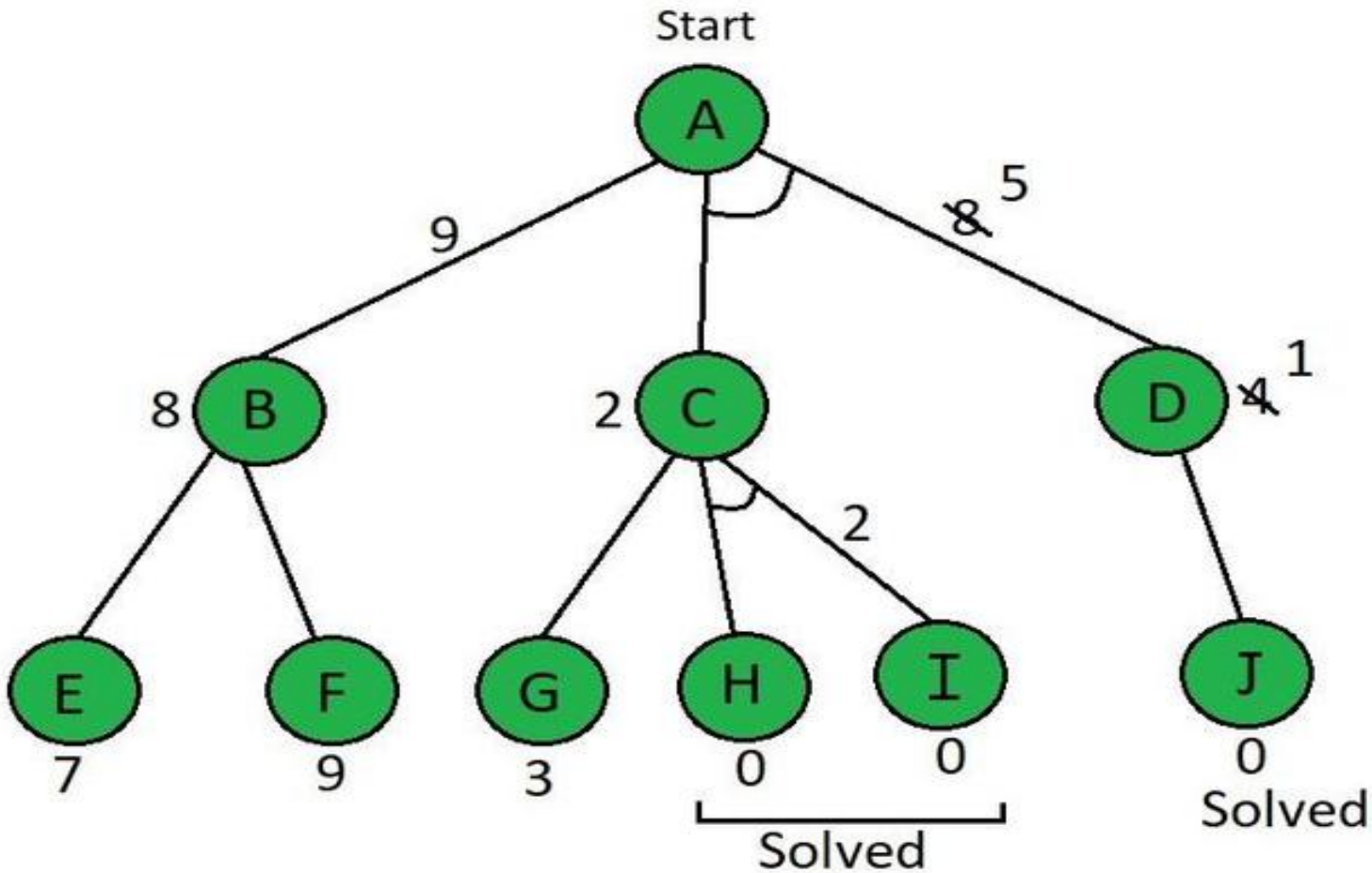
- A* algorithm and AO* algorithm both works on the **best first search**.
- They are both **informed search** and works on given heuristics values.
- **A*** always **gives** the **optimal solution** but AO* doesn’t guarantee to give the optimal solution.
- Once AO* got a solution **doesn’t explore** all possible paths but A* explores all paths.
- When compared to the A* algorithm, the AO* algorithm uses **less memory.**
- opposite to the A* algorithm, the AO* algorithm cannot go into an endless **loop.**

Example:

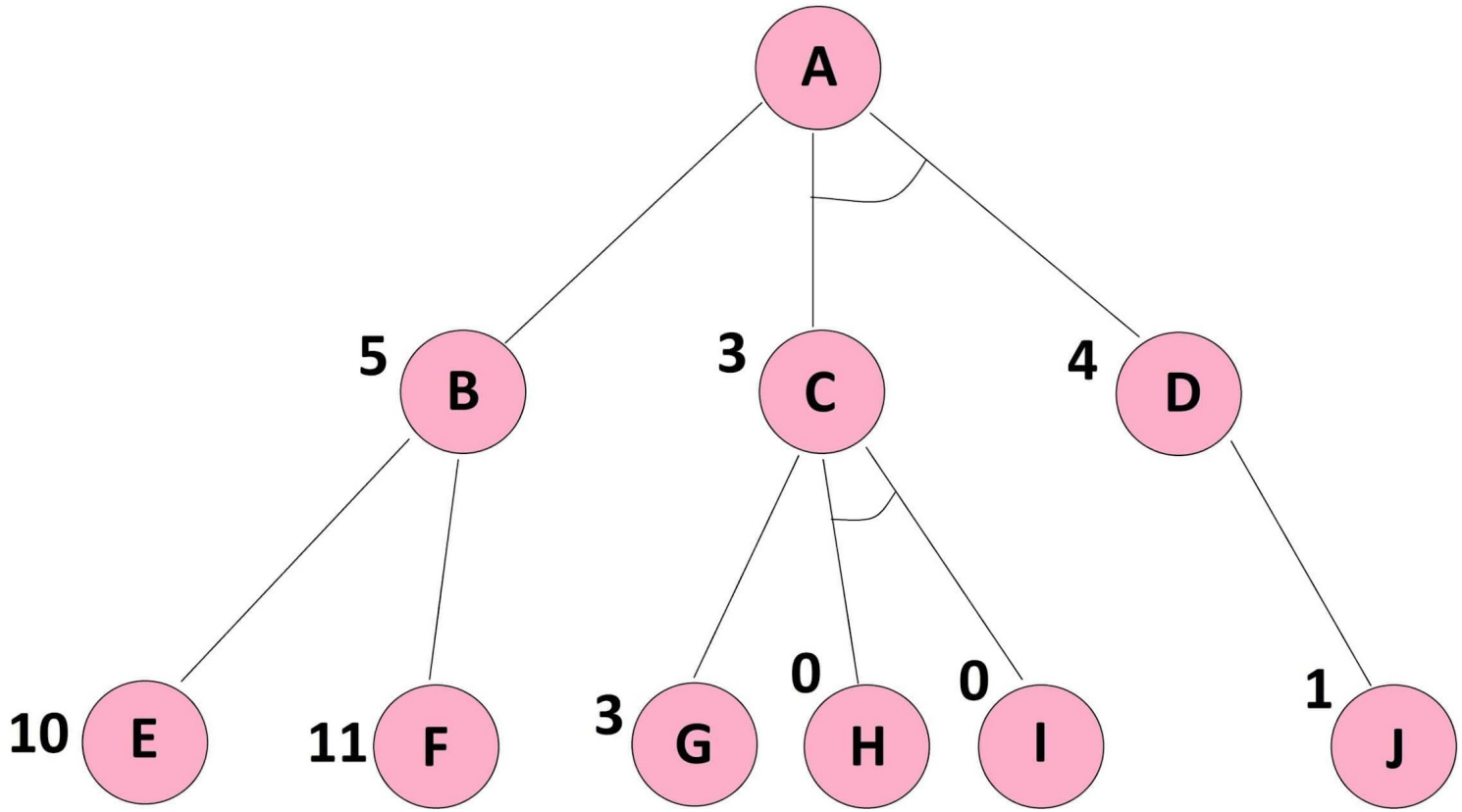


Here in the above example below the Node which is given is the heuristic value i.e h(n). Edge length is considered as 1.

Solution:



Example-2



Content

1. Adversarial Search
2. Elements of Game Playing search
3. Types of algorithms in Adversarial search
4. Minimax

Adversarial Search

- Adversarial search is a **game-playing** technique where the agents are surrounded by a competitive environment.
- A conflicting goal is given to the agents (multiagent).
- These agents compete with one another and try to defeat one another in order to win the game.
- Such conflicting goals give rise to the adversarial search. Here, game-playing means discussing those games where **human intelligence** and **logic factor** is used, excluding other factors such as **luck factor**. **Tic-tac-toe, chess, checkers**, etc., are such type of games where no luck factor works, only mind works.

Techniques required to get the best optimal solution

- **Pruning:** A technique which allows ignoring the unwanted portions of a search tree which make no difference in its final result.
- **Heuristic Evaluation Function:** It allows to approximate the cost value at each level of the search tree, before reaching the goal node.

Elements of Game Playing search

- **S_0:** It is the initial state from where a game begins.
- **PLAYER (s):** It defines which player is having the current turn to make a move in the state.
- **ACTIONS (a):** It defines the set of legal moves to be used in a state.
- **RESULT (s, a):** It is a transition model which defines the result of a move.

- **TERMINAL-TEST (p)**: It defines that the game has ended and returns true.
- **UTILITY (s,p):** It defines the final value with which the game has ended. This function is also known as **Objective function** or **Payoff function**. The price which the winner will get i.e.
- **(-1):** If the PLAYER loses.
- **(+1):** If the PLAYER wins.
- **(0):** If there is a draw between the PLAYERS.

Types of algorithms in Adversarial search

An **adversarial search**, the result depends on the players which will decide the result of the game.

- **Minimax Algorithm**
- **Alpha-beta Pruning.**

Minimax

Minimax is a **decision-making** strategy under **game theory,** which is used to minimize the losing chances in a game and to maximize the winning chances. This strategy is also known as '**Minmax,' 'MM,' or 'Saddle point'.**

MINIMAX Algorithm

MINIMAX algorithm is a backtracking algorithm where it backtracks to pick the best move out of several choices. MINIMAX strategy follows the **DFS (Depth-first search)** concept. Here, we have two players **MIN and MAX,** and the game is played alternatively between them, i.e., when **MAX** made a move, then the next turn is of **MIN.** It means the move made by MAX is fixed and, he cannot change it. The same concept is followed in DFS strategy, i.e., we follow the same path and cannot change in the middle. That's why in MINIMAX algorithm, instead of BFS, we follow DFS.

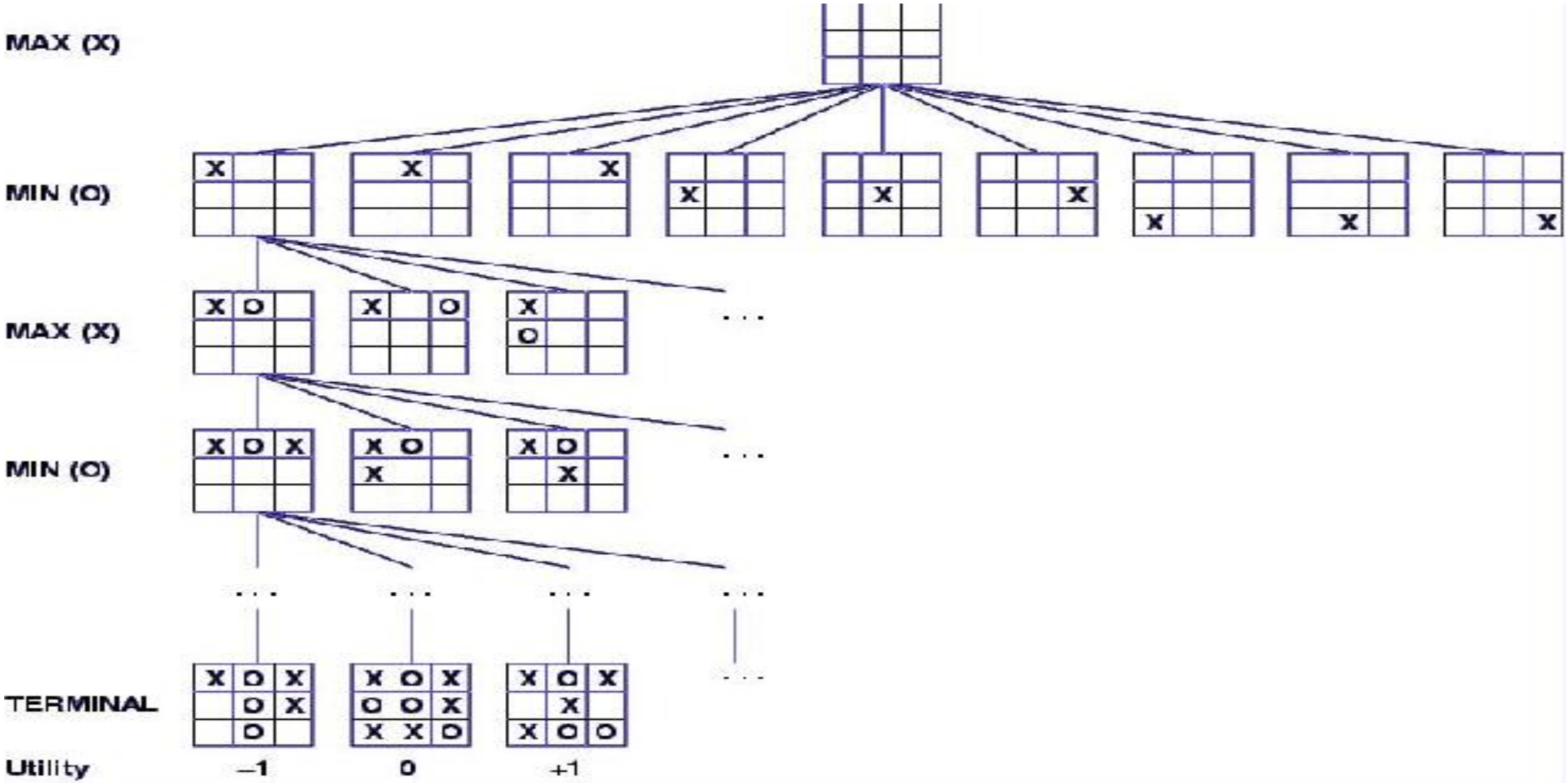
- Keep on generating the game tree/ search tree till a limit **d.**
- Compute the move using a heuristic function.
- Propagate the values from the leaf node till the current position following the minimax strategy.
- Make the best move from the choices.

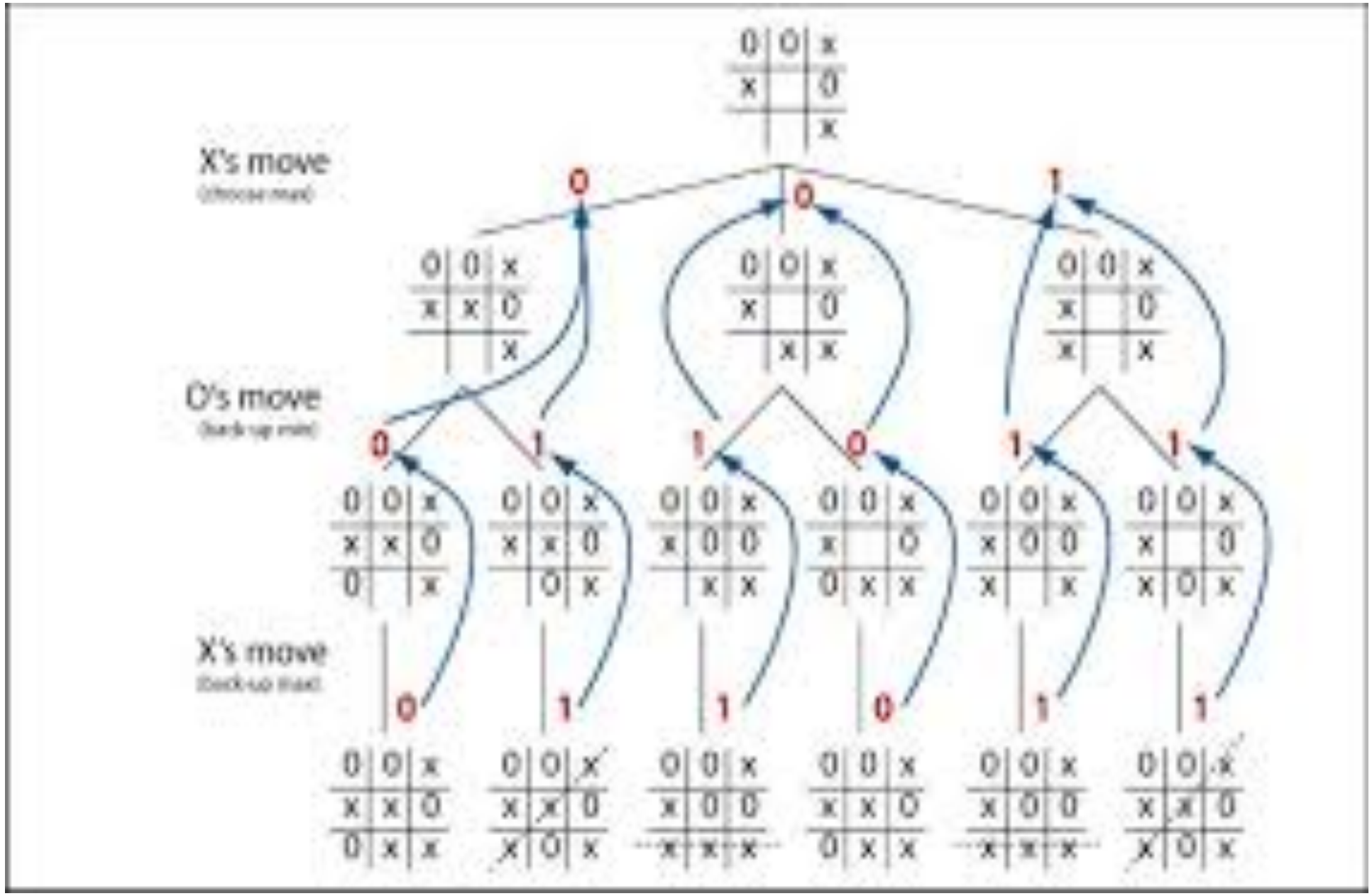
Properties of Mini-Max algorithm

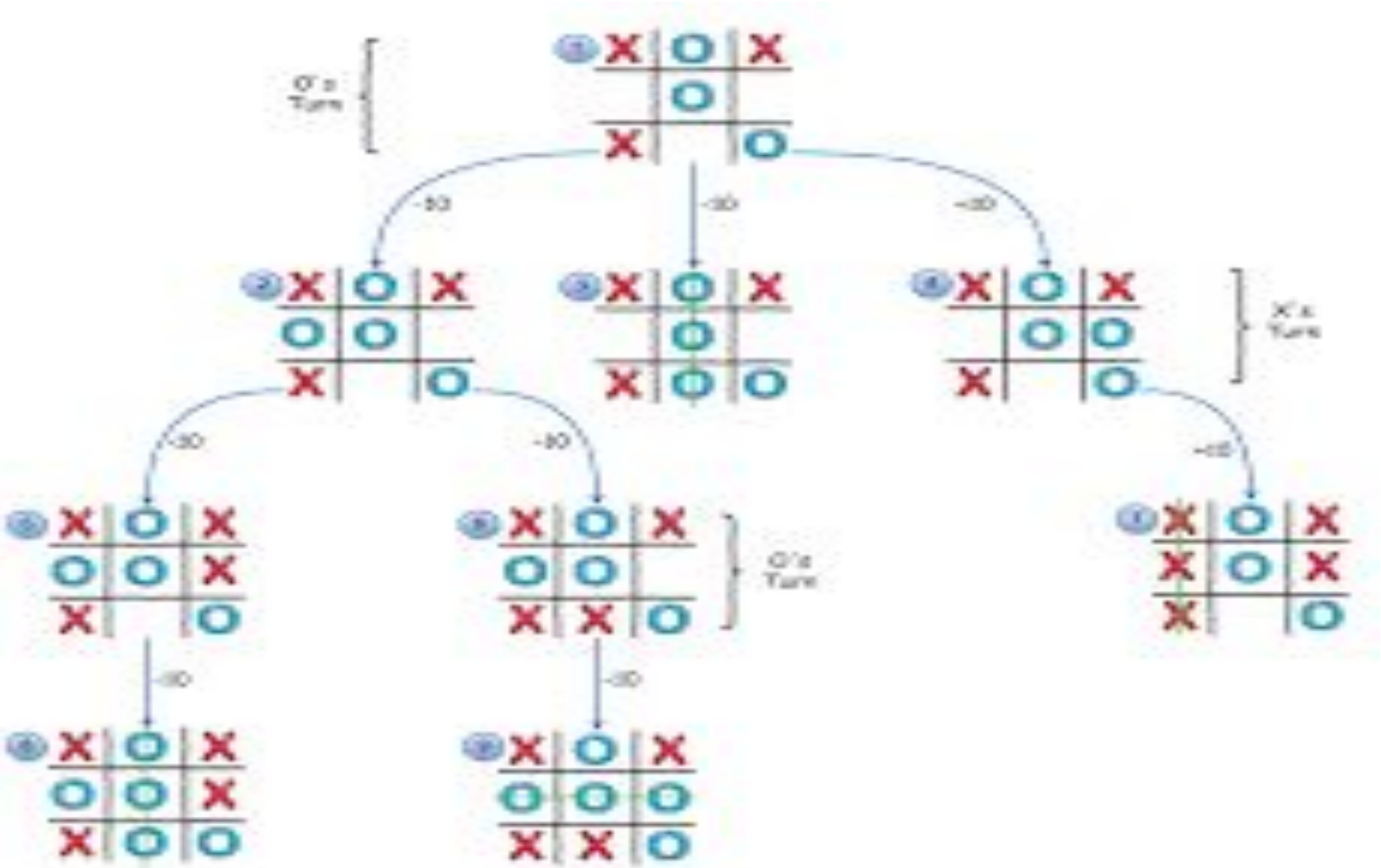
- **Complete-** Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is **$O(b^m)$**, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity-** Space complexity of Mini-max algorithm is also similar to DFS which is **O(bm)**.

Limitation of the minimax Algorithm

- The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc.
- This type of games has a huge branching factor, and the player has lots of choices to decide.
- This limitation of the minimax algorithm can be improved from **alpha-beta pruning** which we have discussed in the next topic.

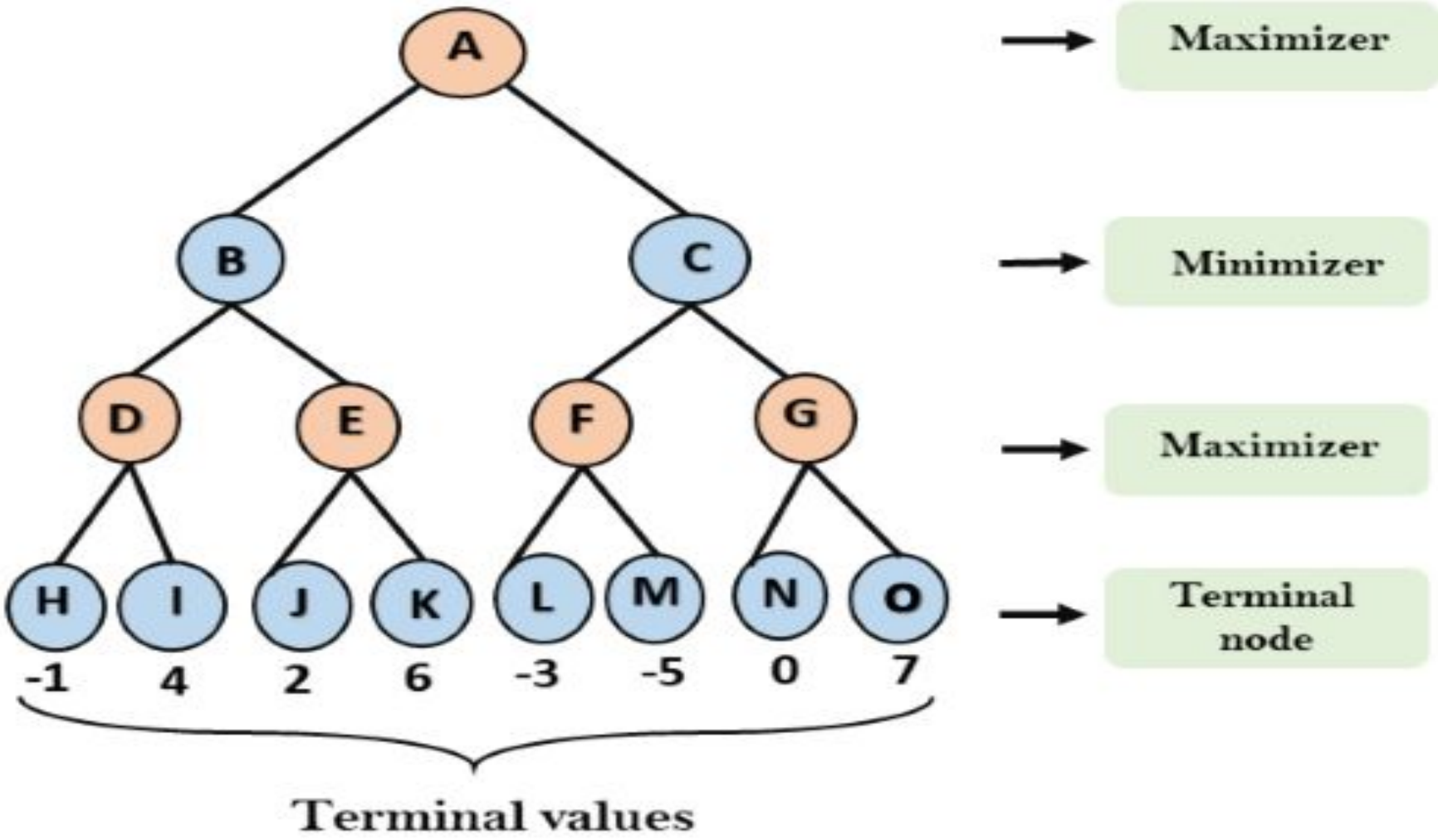

MAX (X)
MIN (O)
MAX (X)
MIN (O)
TERMINAL
Utility
–1
0
+1


X's move
O's move
X's move



Example

- **Step-1:** In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value =- infinity, and minimizer will take next turn which has worst-case initial value = +infinity.


A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Terminal values
Maximizer
Minimizer
Maximizer
Terminal node

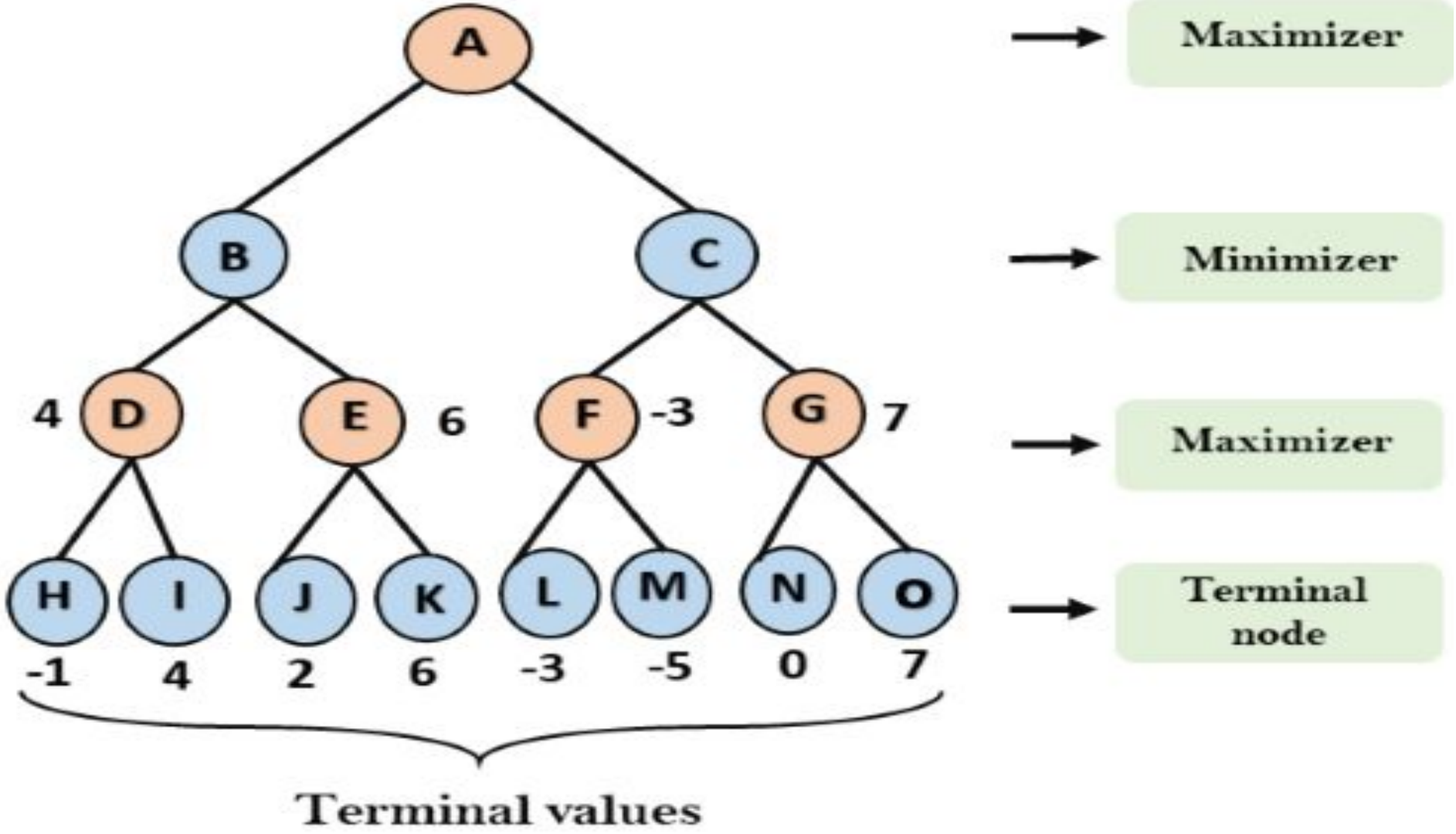
- **Step 2:** Now, first we find the utilities value for the Maximizer, its initial value is -∞, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all.

For node D max(-1,- -∞) => max(-1,4)= 4

For Node E max(2, -∞) => max(2, 6)= 6

For Node F max(-3, -∞) => max(-3,-5) = -3

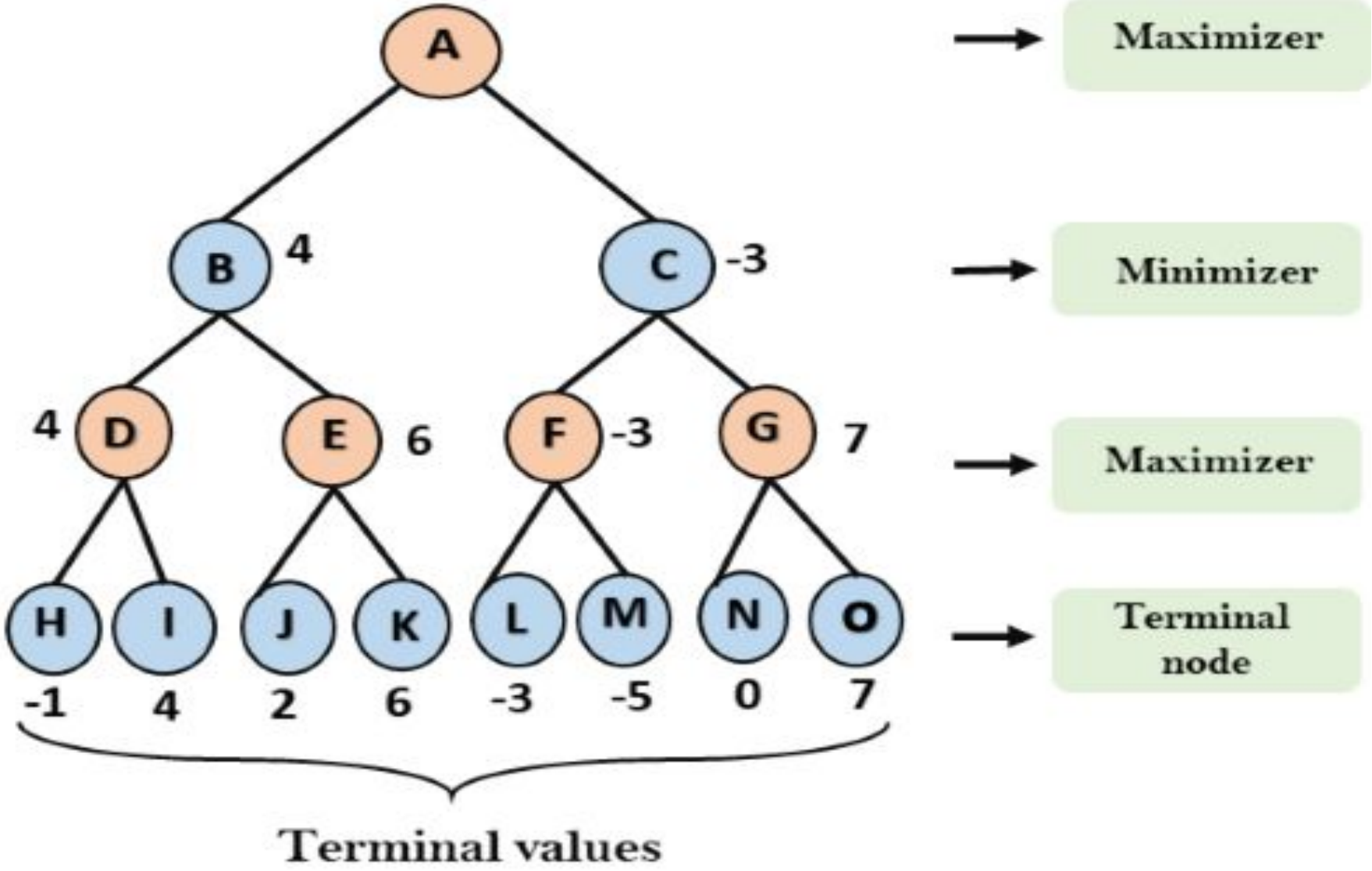
For node G max(0, -∞) = max(0, 7) = 7


A
B
C
4
D
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Terminal values
Maximizer
Minimizer
Maximizer
Terminal node

- **Step 3:** In the next step, it's a turn for minimizer, so it will compare all nodes value with $+\infty$, and will find the 3rd layer node values.

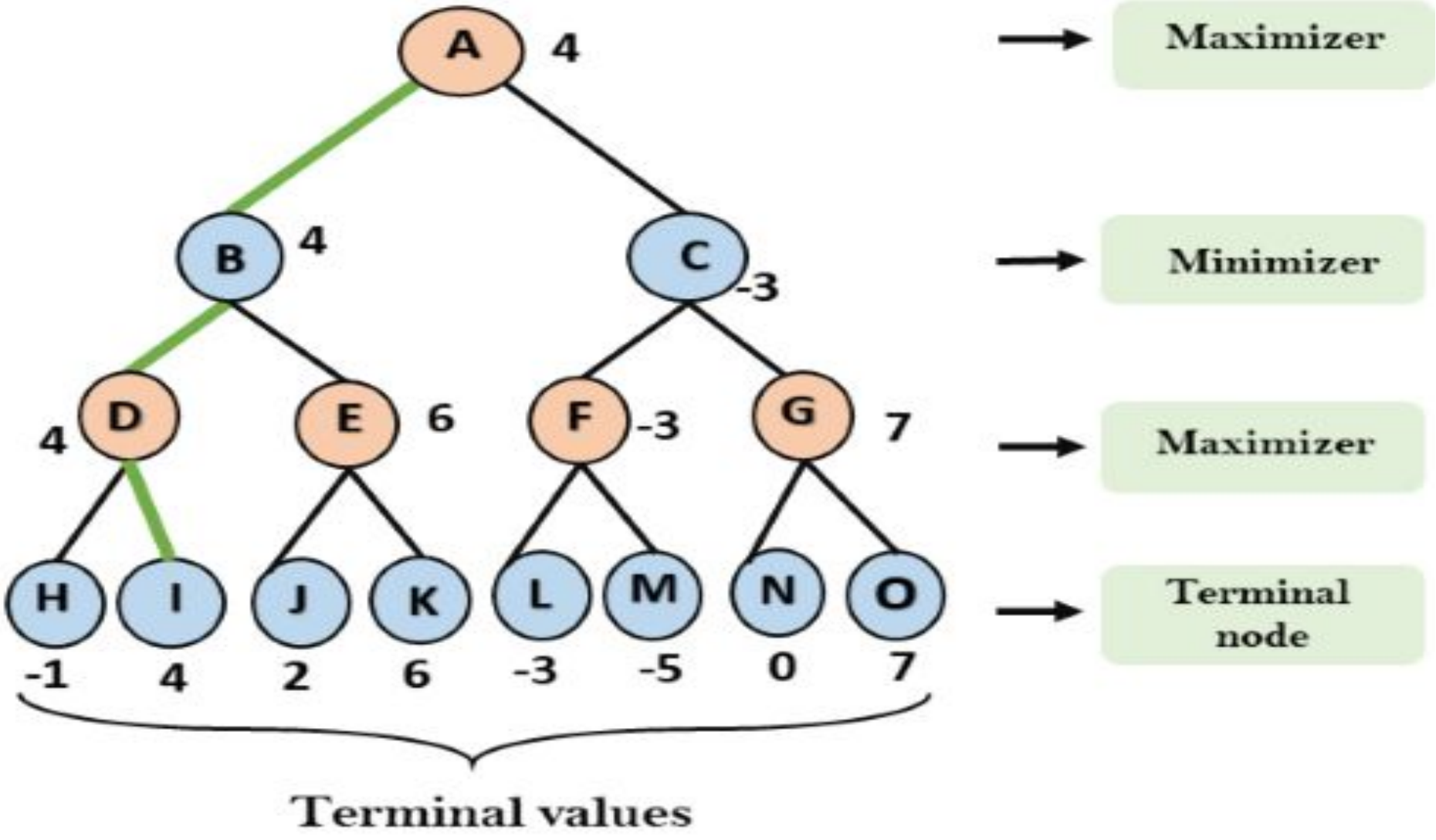
  For node B= min(4,6) = 4

  For node C= min (-3, 7) = -3


A
B
4
C
-3
4
D
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Terminal values
Maximizer
Minimizer
Maximizer
Terminal node

- **Step 4:** Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

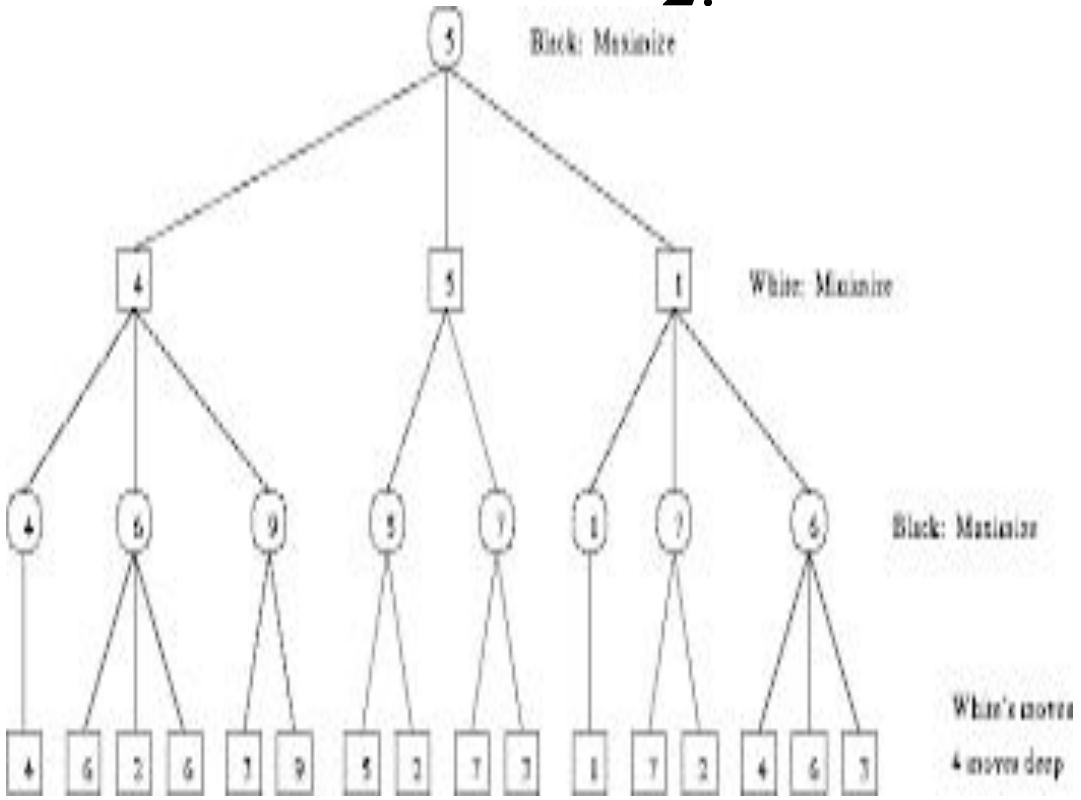
  For node A max(4, -3)= 4


A
4
B
4
C
-3
D
4
E
6
F
-3
G
7
H
I
J
K
L
M
N
O
-1
4
2
6
-3
-5
0
7
Terminal values
Maximizer
Minimizer
Maximizer
Terminal node

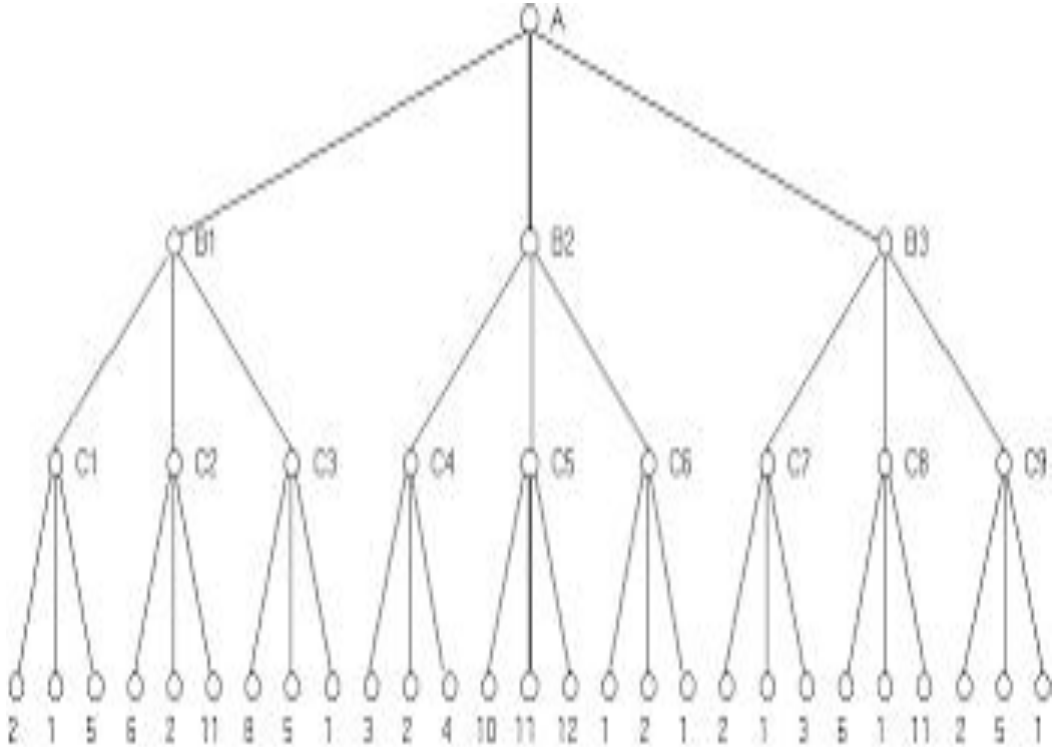
Important Questions

Solved the following figure by mimimax method-

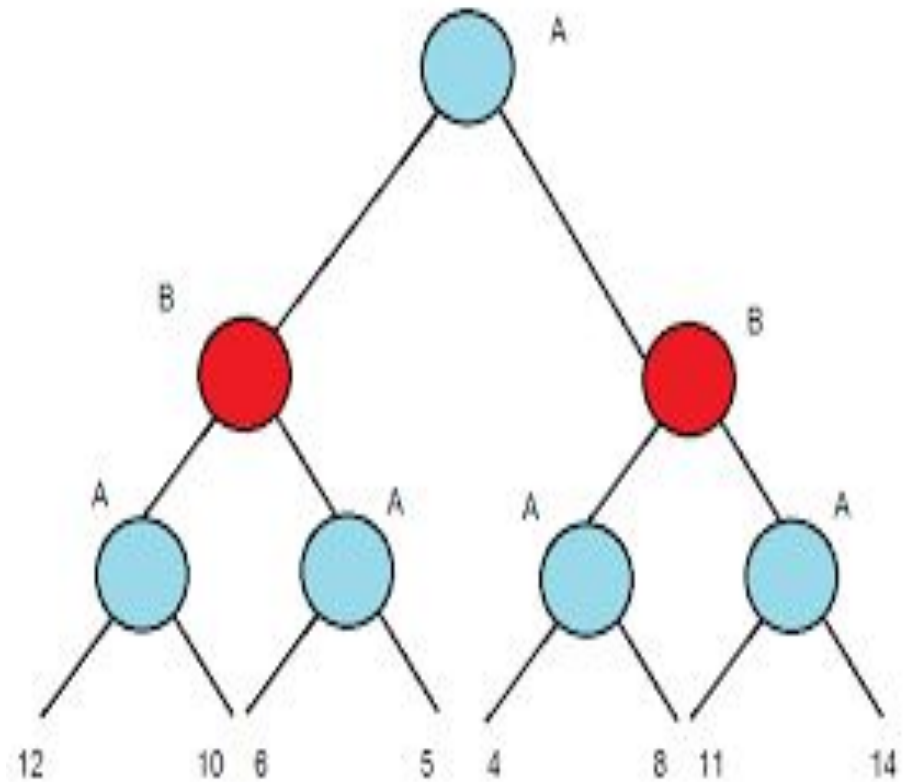
1.



2.

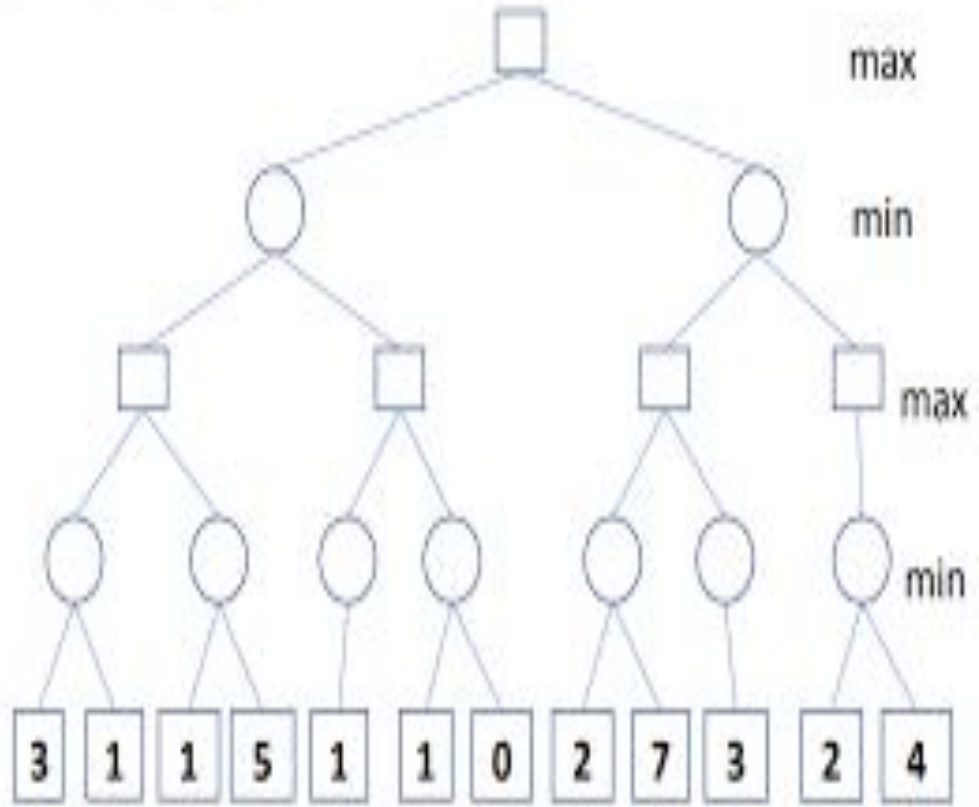


3.



4.

Please follow the Minimax algorithm and put a number in each rectangle and circle in the following game tree. Please use "/" to mark in the tree directly the branches that we need not search (using the alpha-beta pruning algorithm).



References

Text Book:

Stuart Russell, Peter Norvig, “Artificial Intelligence – A Modern Approach”, Pearson Education

Reference Book:

Elaine Rich and Kevin Knight, “Artificial Intelligence”, McGraw-Hill

Other:

https://www.javatpoint.com/ai-alpha-beta-pruning

https://www.mygreatlearning.com/blog/alpha-beta-pruning-in-ai/

https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta- pruning/

Subject Name:-Foundations of Artificial Intelligence Systems

Subject Code:- 21BTCS016

Unit No.:- 2

Lecture No.:- 8

Topic Name :- Alpha-Beta Algorithm

Dr. Arvind Jagtap
Department of CSE

Content

1. Alpha-Beta Pruning
2. Condition for alpha-beta pruning
3. Pseudo-code for Alpha-beta Pruning
4. Working of Alpha-Beta Pruning
5. Example
6. Important Questions

Alpha-Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**. This involves two threshold parameter **Alpha** and **beta** for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.

- The two-parameter can be defined as:
  - **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is **-∞**.
  - **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is +∞.
- The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Condition for Alpha-beta pruning

- The main condition which required for alpha-beta pruning is:

$$\alpha >= \beta$$

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

Pseudo-code for Alpha-beta Pruning

```
function minimax(node, depth, alpha, beta, maximizingPlayer) is
if depth ==0 or node is a terminal node then
return static evaluation of node

if MaximizingPlayer then      // for Maximizer Player
   maxEva= -infinity
   for each child of node do
   eva= minimax(child, depth-1, alpha, beta, False)
  maxEva= max(maxEva, eva)
  alpha= max(alpha, maxEva)
   if beta<=alpha
```

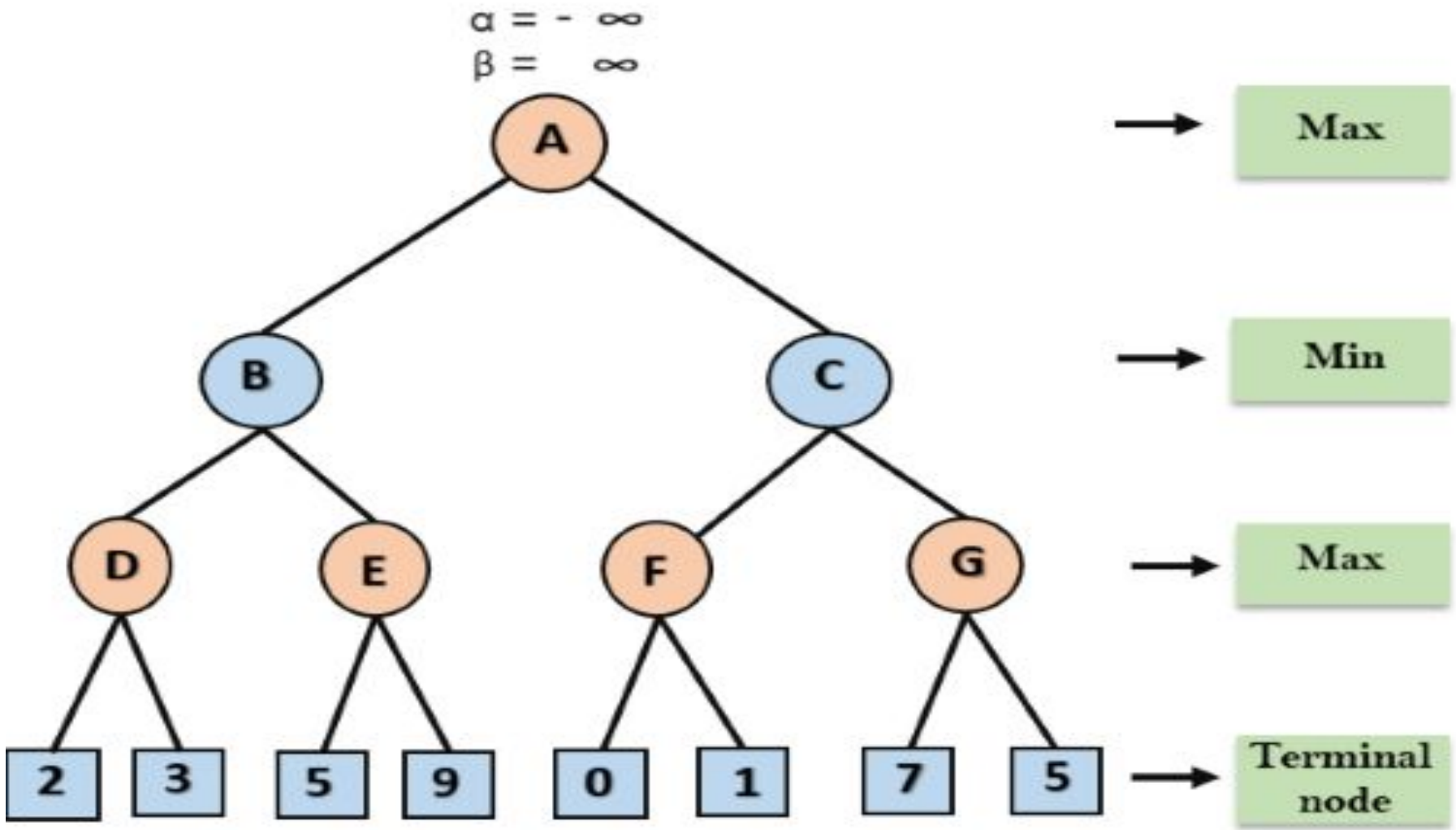
```
 break
 return maxEva
else                          // for Minimizer player
   minEva= +infinity
   for each child of node do
   eva= minimax(child, depth-1, alpha, beta, true)
   minEva= min(minEva, eva)
   beta= min(beta, eva)
    if beta<=alpha
  break
 return minEva
```

Working of Alpha-Beta Pruning

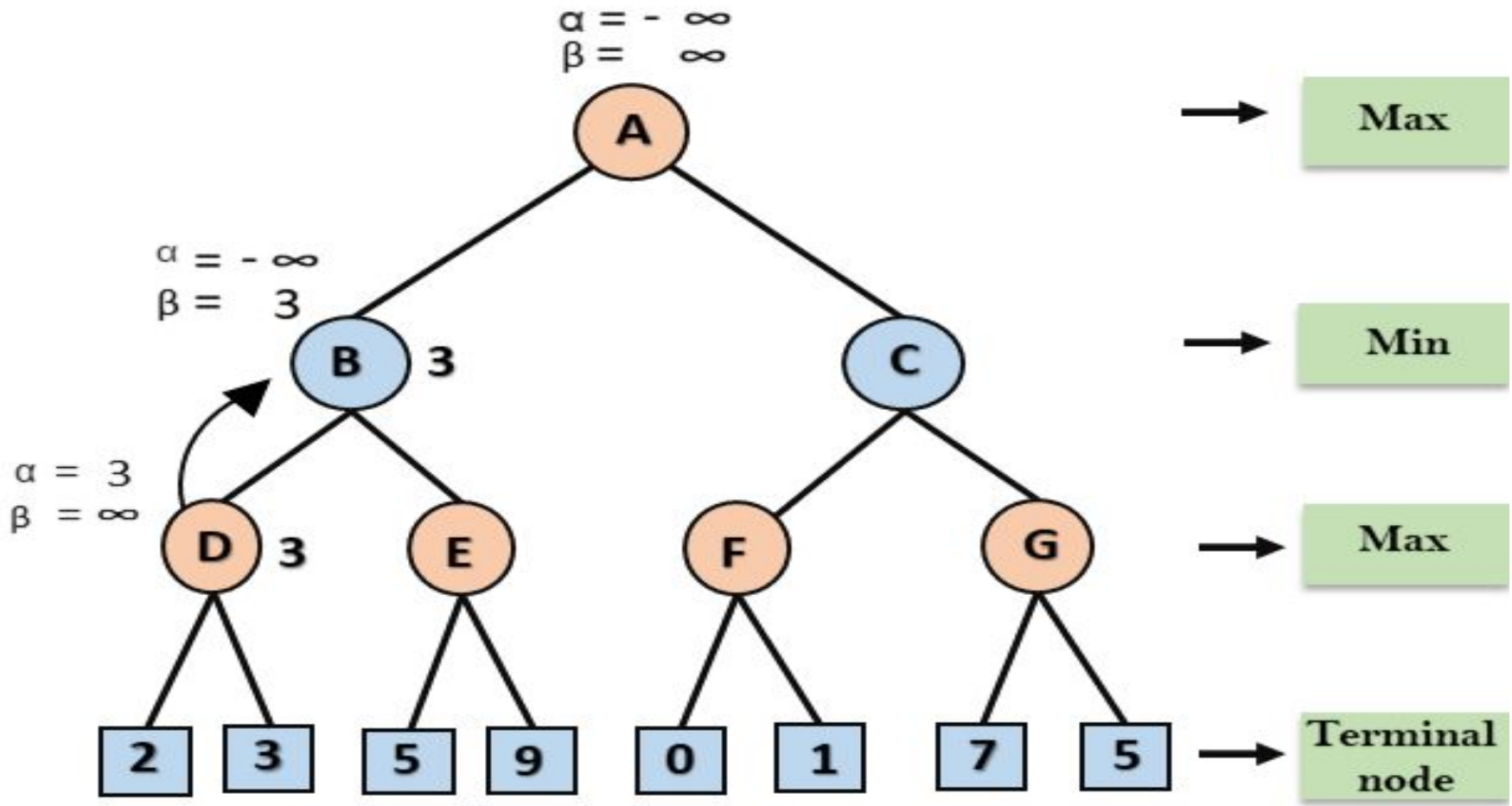
Let's take an example of two-player search tree to

understand the working of Alpha-beta pruning:

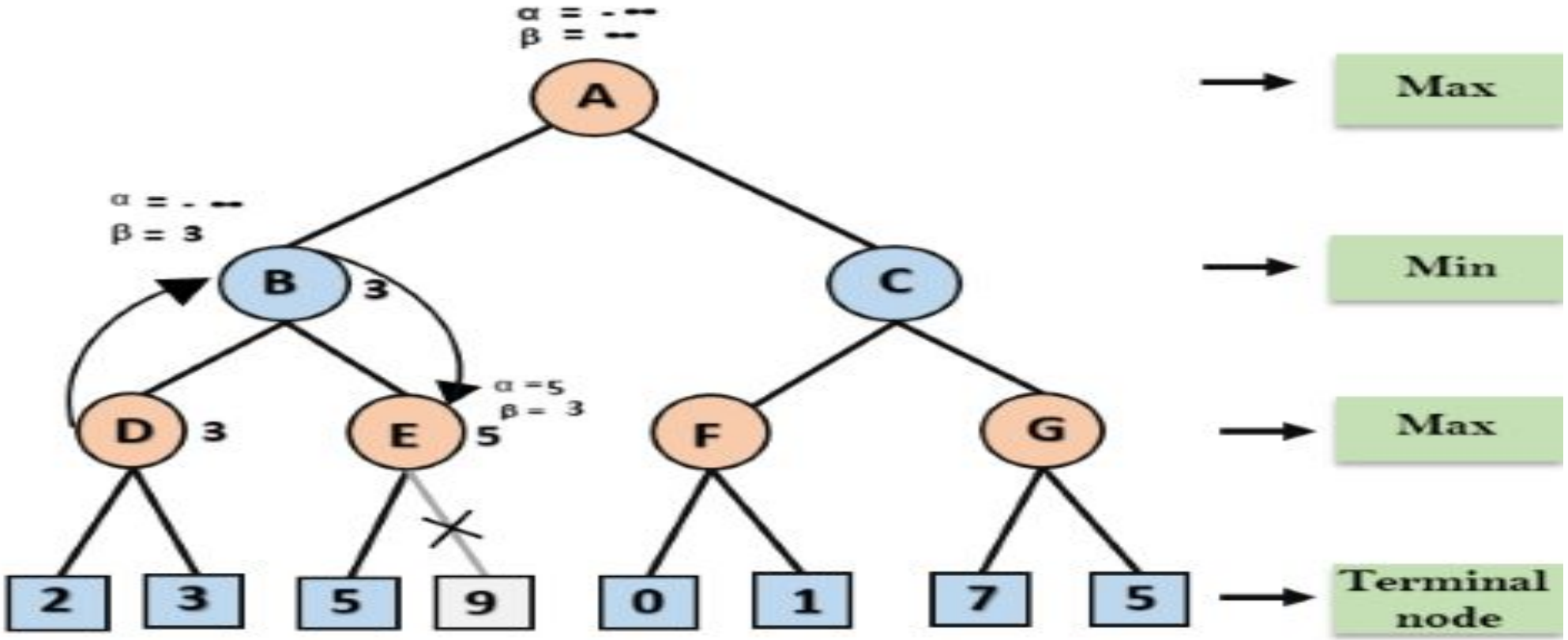
- **Step 1:** At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$, these value of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.


α = - ∞
β = ∞
A
B
C
D
E
F
G
2
3
5
9
0
1
7
5
Max
Min
Max
Terminal node

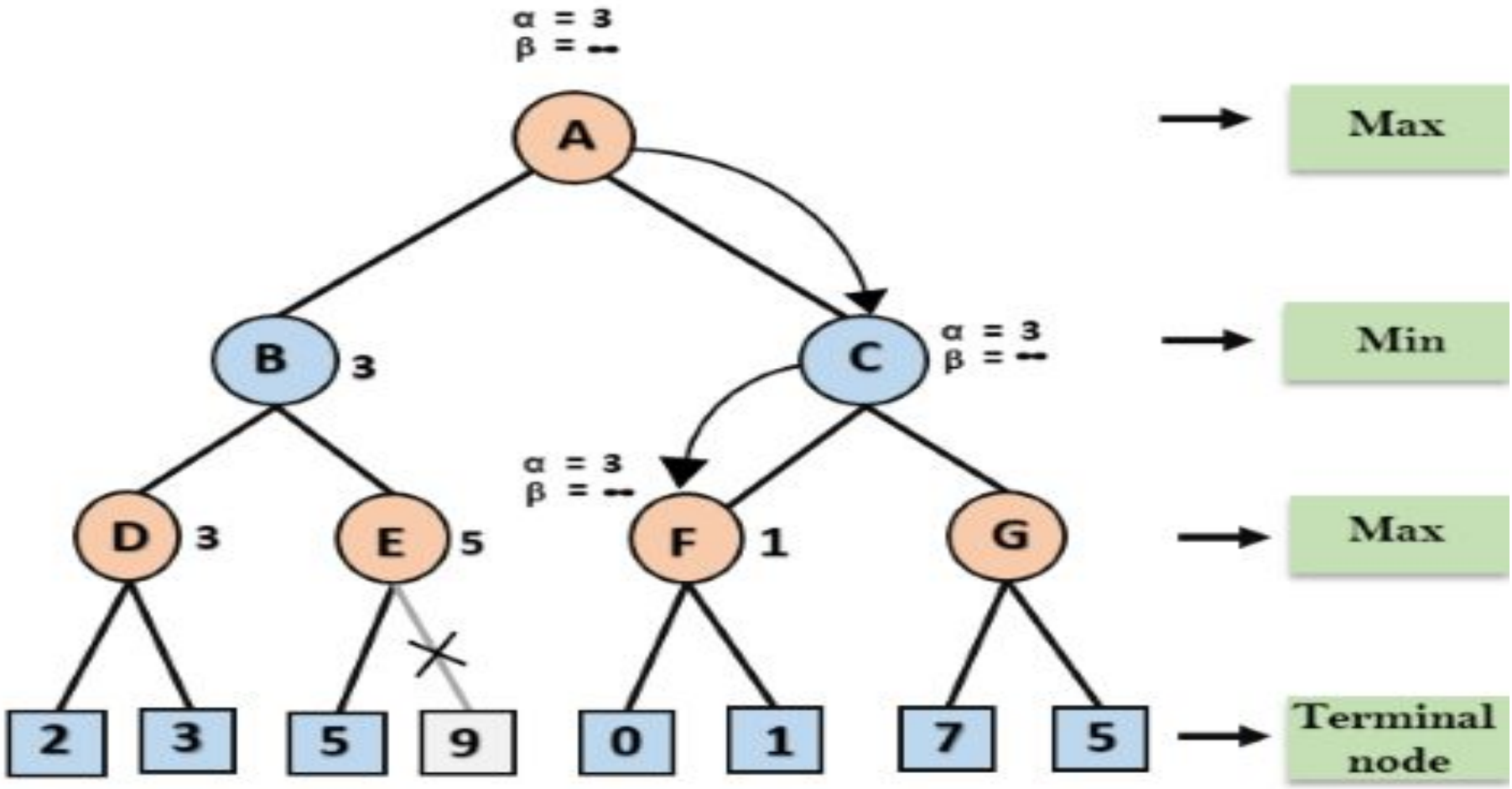
- **Step 2:** At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.
- **Step 3:** Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.


α = - ∞
β = ∞
A
Max
α = - ∞
β = 3
B
3
C
Min
α = 3
β = ∞
D
3
E
F
G
Max
2
3
5
9
0
1
7
5
Terminal node

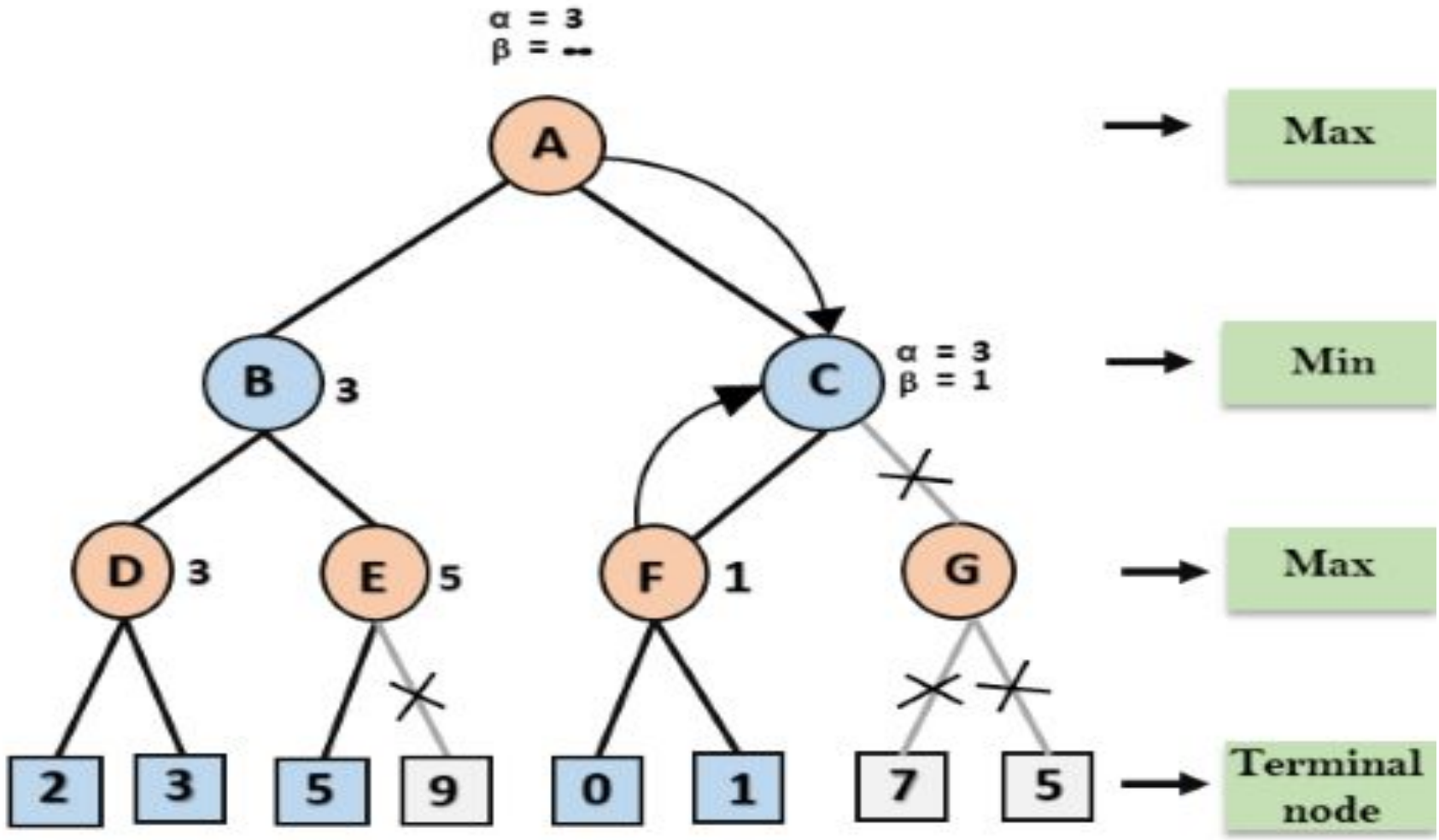
- In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.
- **Step 4:** At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.


α = -∞
β = +∞
A
Max
α = -∞
β = 3
B
3
C
Min
α =5
β = 3
D
3
E
5
F
G
Max
2
3
5
9
0
1
7
5
Terminal node

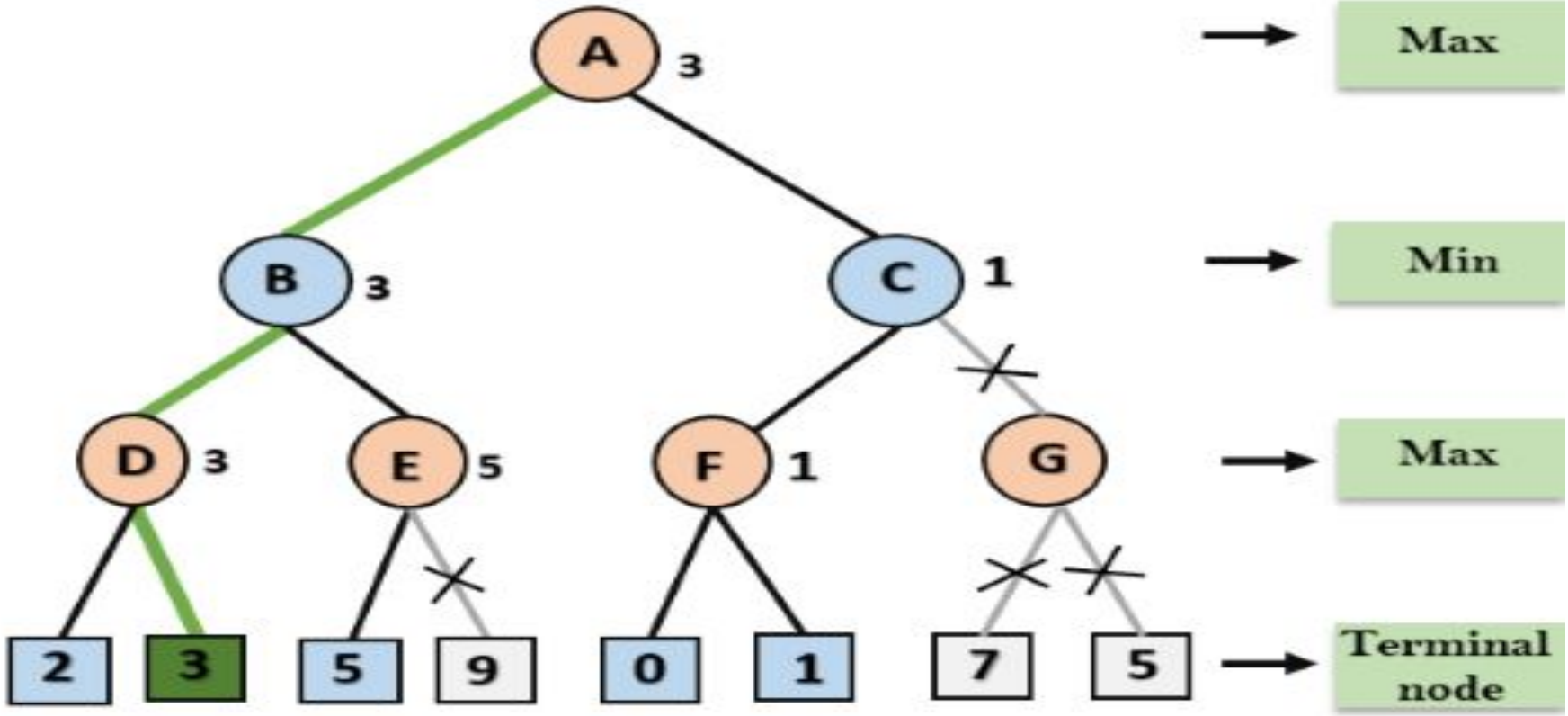
- **Step 5:** At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.
- At node C, α=3 and β= +∞, and the same values will be passed on to node F.
- **Step 6:** At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.


α = 3
β = ∞
A
Max
B
3
C
α = 3
β = ∞
Min
α = 3
β = ∞
D
3
E
5
F
1
G
Max
2
3
5
9
0
1
7
5
Terminal node

- **Step 7:** Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.


α = 3
β = ∞
A
B
3
C
α = 3
β = 1
D
3
E
5
F
1
G
2
3
5
9
0
1
7
5
Max
Min
Max
Terminal node

- **Step 8:** C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.


A
3
B
3
C
1
D
3
E
5
F
1
G
2
3
5
9
0
1
7
5
Max
Min
Max
Terminal node

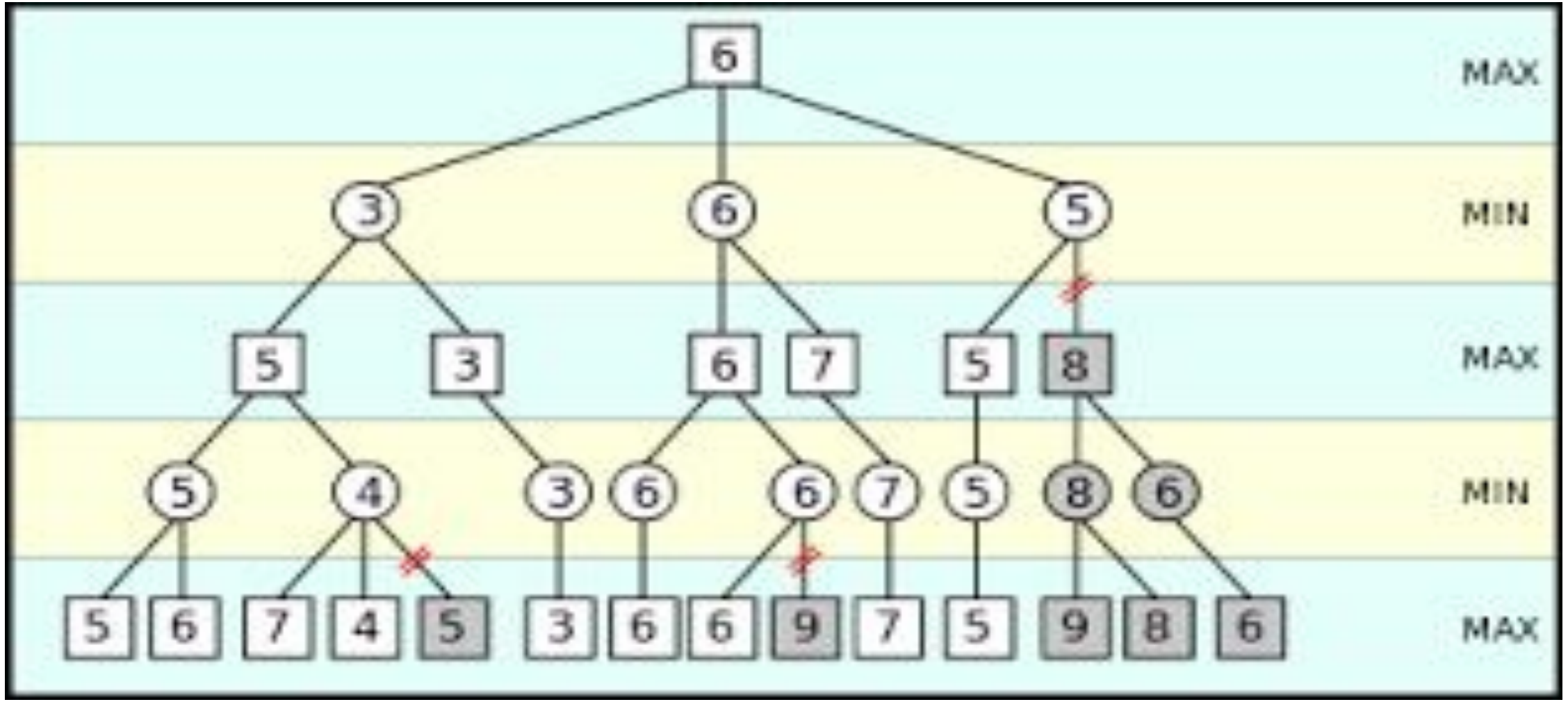
Move Ordering in Alpha-Beta pruning

The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning. It can be of two types:

- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(b^m)$.

- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is $O(b^{m/2})$.

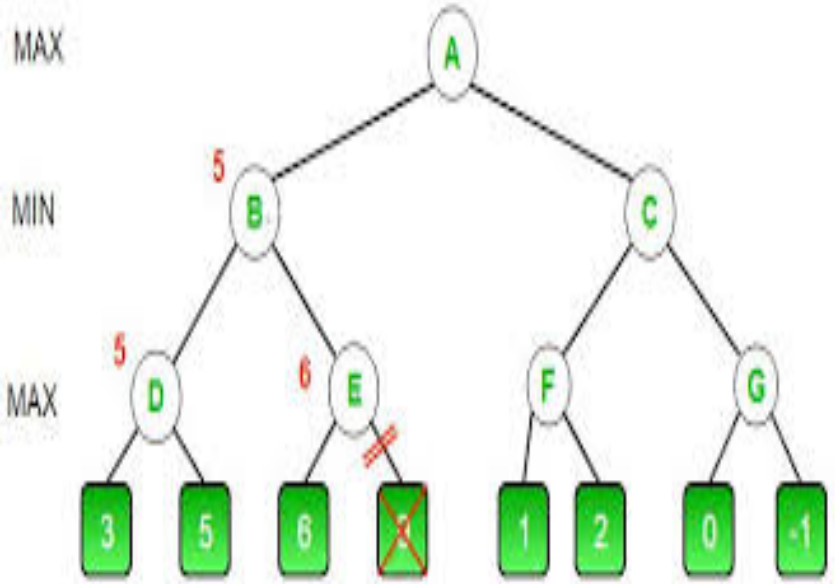
Example



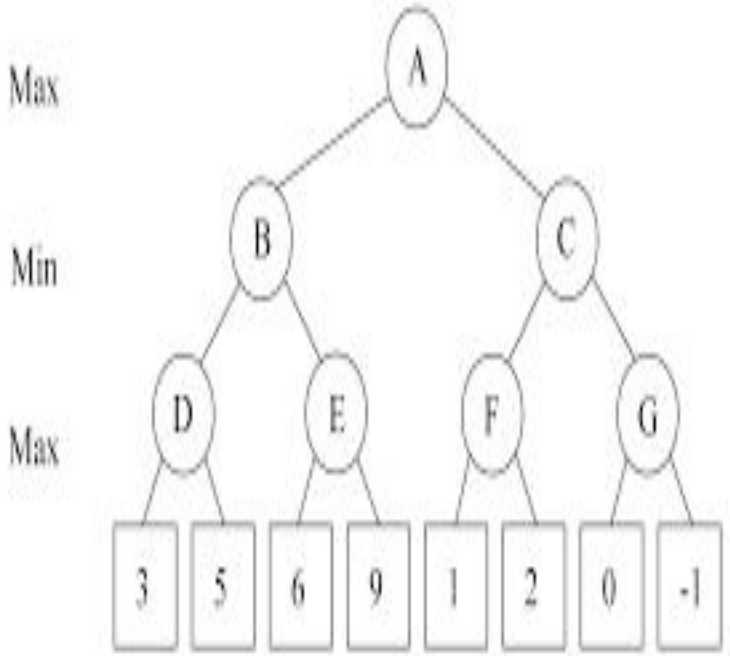
Important Questions

Solved following question by alpha-beta pruning method-

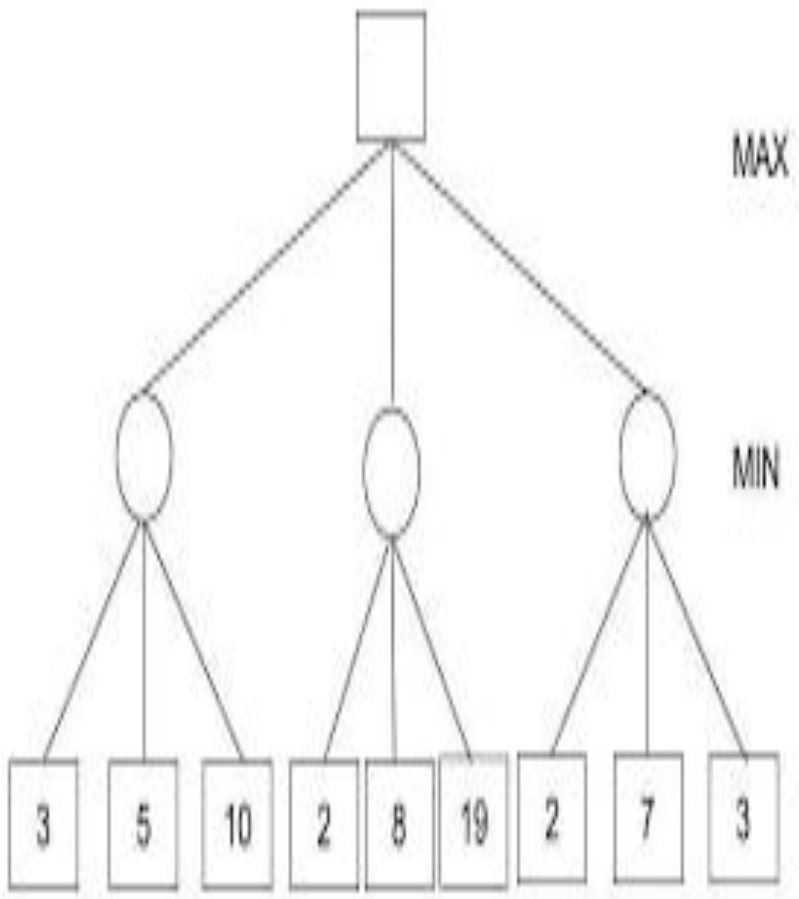
1.



2.



3.



References

Text Book:

Stuart Russell, Peter Norvig, “Artificial Intelligence – A Modern Approach”, Pearson Education

Reference Book:

Elaine Rich and Kevin Knight, “Artificial Intelligence”, McGraw-Hill

Other:

https://www.javatpoint.com/ai-alpha-beta-pruning

https://www.mygreatlearning.com/blog/alpha-beta-pruning-in-ai/

https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta- pruning/

Thank You